

40 种 Jac Matmul : 走向 GPU 的功能地图

算法如何计算，OSP 如何表达，GPU 后端还缺什么

40

逐项解释 · 最低运行能力 · 高性能扩展

6 种循环顺序 + 19 种算法 / 表达 + 7 种 OSP + 8 条库路径

统一问题： $A[M,K] \times B[K,N] \rightarrow C[M,N]$ ，当前样例使用 float64

代码快照：[addef5acb](#) | 分支：[paper/jac-on-gpu](#)

交付范围：分析与 PDF。保持现有算法和 GPU 后端代码不变。

查阅功能定义：[第 63 页导航](#)，[第 64-84 页展开说明](#)。

源码：[jac/examples/matmul/catalog.jac](#)

先读结论与证据

这份 **slides** 的结论

- 最通用的缺口是设备 buffer/view、动态索引与循环，以及明确的输出任务映射。
- 保留 OSP 形式时，双游标/邻居/边属性需要独立的图语义扩展；CSR 上传本身不够。
- 高效 dense GEMM 还需 tile 协作与数据复用，或走 GPU 库互操作路线。

证据范围与状态含义

- validation_results.json : $40 \times 19 = 760$ 次 CPU 比较；13 组精确、6 组浮点容差，另有 90 次配置检查。
- 已核对记录中的 8 个 Jac 源码 SHA256 与当前文件一致；该记录明确 `gpu_executed=false`。
- 逐项 GPU 状态是基于现有选择器/ABI 的源码分析；已有链式 GPU 示例单独记录，其他条目不声称实机通过。

能力状态以源码快照 addef5acb 为准；判断目标是保留各实现主要布局/语义，而非统一改造成乘数链。

当前 Jac On GPU 已经有什么

语言与执行模型

- 一个 entry ability ; 读取 here 的固定标量字段 , 更新 self 的私有标量状态。
- int64 或 float64 , 可混 bool ; 固定 plain obj 的标量叶子可展开为 SoA。
- 标量算术、比较、布尔逻辑、if/elif/else ; 尾部 visit 沿单一空属性边类型。
- 链式或实验性 CSR FIFO 遍历 ; 每个 GPU 线程独立完成一个 walker。

内存与已知限制

- 共享只读图、字段 SoA、节点/起点重映射 ; 设备只存标量和整数索引。
- 独立结果列 ; 有界标量 report 采用 slot×walker_count+lane 排列 , bool 有类型标签。
- 同一 numeric schema 不能混 int 与 float ; 无数组字段、一般循环、节点句柄状态、普通查询循环或边 payload。
- 每次 batch 仍重新打包/上传 ; 容量复用不等于输入长期驻留。

依据 : ptx_walker.jac::project_fields / chain_expr / select_chain_walkers ; ptx_csr.jac ; scripts/gpu_walker/runtime.md。

八类能力：后文功能标签的含义

区分最低可执行要求与优化；这是建议的能力分类，当前 Jac 并没有这些新 API。

D

设备数据

固定 dtype buffer、tensor view、shape/stride、只读/可写、生命周期。

P

任务与结果所有权

grid/batch 映射、独占输出、隐式坐标任务、无 visit 的 compute entry。

G

图语义

node/edge handles、第二 receiver、邻居/边查询、异构 schema、edge SoA。

L

GPU 互操作

host FFI、GPU tensor ABI、库 handle、context/stream、错误与完成语义。

I

设备控制与寻址

int 索引 + float 算术、for/while、局部变量、边界处理、helper lowering。

R

归约与阶段依赖

partial 合并、分层 reduction、多 kernel pipeline、workspace、冲突策略。

H

GPU 协作与内存层次

warp/block、shared memory、barrier、寄存器 tile；异步复制 / MMA 可选。

N

数值契约

dtype/精度、求和顺序、FMA/reassociation、确定性与容差。

图、walker、report 的设备内存布局

现有结构示意：逻辑对象变成列；新增 tensor view 可以接入同样的 arena 思路。

Graph SoA

left[N]

right[N]

next[N] / CSR

...

Walker SoA

heads[W]

initial[S,W]

results[S,W]

status[W]

Reports

slot0: w0 w1 w2...

slot1: w0 w1 w2...

tags / counts

...

同一 slot 的 w0、w1 report 相邻；同一 walker 的两个 slot 相隔 W 个元素。

数组起始处按 256 字节对齐；bool 当前用 8 字节存储。结构化报告整体不是通用 tensor 输出 ABI。

自动合并访存：布局必须与 **lane** 调度一起决定

以下固定 i 、 k ，lane 沿 j ；假设元素 row-major，warp 内读取同一条 load 指令。

读取 **B**

$B[k, j + \text{lane}]$ ：相邻元素



读取 **A**

$A[i, k]$ ：相同地址，可共享/缓存



写入 **C**

$C[i, j + \text{lane}]$ ：相邻输出



如果改为 lane 沿 i ， A/C 地址通常跨行 stride；SoA 或“先转置”不能单独保证 coalescing。

编译器需要访问映射、读写依赖和布局选择；任意图遍历的下一跳仍可能分散。

GPU 最低运行与高性能是两层能力

最低可执行：先定义清楚

- 一线程一个 $C[i,j]$ ；K 循环串行，A/B 共享只读，C 唯一写入。
- 需要设备 buffer、整数索引、一般循环、局部 float 累加器、launch 与 output store。
- 无需 Tensor Core、shared memory、原子操作或 device recursion，也能正确完成 matmul。

高性能：逐层加入

- 输出 tile 映射到 block/warp；协作加载输入到 shared memory，以 barrier 保证可见性。
- 每线程寄存器 tile 复用输入；再做流水线、布局调优和输出阶段的连续存储。
- MMA / Tensor Core 是 dtype、硬件和精度相关的可选后端；当前 float64 不能无条件改成低精度。

性能结构参考 NVIDIA CUTLASS Efficient GEMM（可点击）；Jac 能力映射与优先级为对仓库代码的工程推断。

数值一致：需要的是契约，不只是一个 **sum**

已验证的 **CPU** 范围

- 13 组小整数数据对独立整数参考精确相等；6 组浮点数据对 Decimal 100 位参考。
- float64 样例使用 $\text{rel_tol}=1\text{e-}10$ 、 $\text{abs_tol}=1\text{e-}10$ ；记录最大绝对误差约 $1.34\text{e-}14$ 。
- 有限实数、非空矩形输入；此记录不证明极端数值、溢出或所有矩阵均稳定。

GPU 必须明确的行为

- 串行 K、pairwise、split-K、Kahan、fsum、Strassen 的舍入路径不同；不能统一声称逐位相同。
- FMA 融合、reassociation 和 Tensor Core 精度选择要有声明；Kahan 的补偿表达式尤其不能被消除。
- 并行 scatter 的原子顺序可能改变；确定性需求应使用固定分组/树，并重新做 CPU/GPU 差分。

证据：validation_results.json；现有 CPU 输出一致指的是该测试范围内的精确值或指定容差。

CPU 与 GPU 的边界：不必把整个函数搬到设备

最低 GPU 运行可保留 host 准备；低开销路径再减少构图、重上传和中间数组往返。



递归可由 host 展开；临时空间可预分配；查询的实体可用 device handle；输出可由 lane 生成坐标。

保留逻辑 node/edge 的表达力，不要求显式创建所有乘法节点或给每个节点独立分配显存。

建议设计是新增能力方向，不是现有 Jac 语法/API 已经实现的承诺。

40 项覆盖清单：每项在后文有独立说明页

编号对应实现页；不同接口与不同算法分别解释，不把 40 项说成 40 个独立的底层数值算法。

01 loop_ijk

02 loop_ikj

03 loop_jik

04 loop_jki

05 loop_kij

06 loop_kji

07 transposed_dot

08 comprehension

09 flat_buffers

10 blocked

11 packed_tiles

12 register_2x2

13 split_k

14 pairwise

15 compensated

16 accurate_sum

17 winograd_pairs

18 threaded_rows

19 overloaded_operator

20 recursive_blocks

21 strassen

22 strassen_winograd

23 csr_dense

24 csr_csc_merge

25 coo_scatter

26 product_chains

27 dual_cursors

28 row_nodes

29 output_cell_tasks

30 output_tile_tasks

31 bipartite_join

32 edge_payloads

33 numpy_operator

34 numpy_matmul

35 numpy_dot

36 numpy_einsum

37 numpy_tensordot

38 numpy_broadcast

39 scipy_blas

40 scipy_sparse

01 loop_ijk

需要 GPU 扩展

逐输出点积

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- 每个 (i,j) 完整执行 K 点积；C 的单元可由一个线程独占。
- 算术量 $O(MNK)$ ，结果 $O(MN)$ ；不需复制乘数对或显式构图。
- 以 (i,j) 为工作项；保留串行 k，即可避免原子累加。

计算示意（不是新增 Jac API）

```
for i in range(M):
    for j in range(N):
        for k in range(K):
            C[i,j] += A[i,k] * B[k,j]
```

HOST 边界

形状检查、设备分配和最终二维列表封装可留在 CPU。

Jac On GPU：最低需要

- D：共享只读 A/B 设备缓冲区；独立可写 C，支持 shape / stride。
- I：整数动态下标、一般 for 循环、局部浮点累加器；索引类型与数值类型分离。
- P：定义 i/j 的 GPU 任务映射和唯一写入者；若并行 k，另需 R 归约。

性能 / 可选扩展

j 方向的相邻线程在同一 k 读取相邻 B[k,j]，A 可广播或复用。

已有能力与缺口

当前已有标量乘加；此函数的数组循环尚未被 GPU walker 后端支持。

02 loop_ikj

需要 GPU 扩展

行内外积更新

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出

如何计算 / 数据表示

- 固定 i、k 后沿 j 更新整行；A[i,k] 被一组列复用。
- 算术量 $O(MNK)$ ，结果 $O(MN)$ ；不需复制乘数对或显式构图。
- 可让一组线程拥有一行/行块；各线程只写自己的列。

计算示意（不是新增 Jac API）

```
for i in range(M):
    for k in range(K):
        for j in range(N):
            C[i,j] += A[i,k] * B[k,j]
```

HOST 边界

形状检查、设备分配和最终二维列表封装可留在 CPU。

Jac On GPU：最低需要

- D：共享只读 A/B 设备缓冲区；独立可写 C，支持 shape / stride。
- I：整数动态下标、一般 for 循环、局部浮点累加器；索引类型与数值类型分离。
- P：定义 i/j 的 GPU 任务映射和唯一写入者；若并行 k，另需R 归约。

性能 / 可选扩展

j 映射到相邻线程更符合行优先 B/C 的连续访问。

已有能力与缺口

当前已有标量乘加；此函数的数组循环尚未被 GPU walker 后端支持。

03 loop_jik

需要 GPU 扩展

按列组织点积

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出

如何计算 / 数据表示

- 先枚举列，再枚举行；每个 $C[i,j]$ 仍是独立点积。
- 算术量 $O(MNK)$ ，结果 $O(MN)$ ；不需复制乘数对或显式构图。
- GPU 可重排工作项，让 lane 沿 j ；计算语义无需改变。

计算示意（不是新增 Jac API）

```
for j in range(N):
    for i in range(M):
        for k in range(K):
            C[i,j] += A[i,k] * B[k,j]
```

HOST 边界

形状检查、设备分配和最终二维列表封装可留在 CPU。

Jac On GPU：最低需要

- D：共享只读 A/B 设备缓冲区；独立可写 C，支持 shape / stride。
- I：整数动态下标、一般 for 循环、局部浮点累加器；索引类型与数值类型分离。
- P：定义 i/j 的 GPU 任务映射和唯一写入者；若并行 k ，另需 R 归约。

性能 / 可选扩展

若把相邻 i 直接映射到 lane，行优先 A/C 会出现跨行跨度。

已有能力与缺口

当前已有标量乘加；此函数的数组循环尚未被 GPU walker 后端支持。

04 loop_jki

需要 GPU 扩展

列内逐 K 更新

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出

如何计算 / 数据表示

- 固定 j、k 后沿 i 更新一列；B[k,j] 在多行之间复用。
- 算术量 $O(MNK)$ ，结果 $O(MN)$ ；不需复制乘数对或显式构图。
- 按输出行/列建立唯一写入者；k 保持串行或显式归约。

计算示意（不是新增 Jac API）

```
for j in range(N):
    for k in range(K):
        for i in range(M):
            C[i,j] += A[i,k] * B[k,j]
```

HOST 边界

形状检查、设备分配和最终二维列表封装可留在 CPU。

Jac On GPU：最低需要

- D：共享只读 A/B 设备缓冲区；独立可写 C，支持 shape / stride。
- I：整数动态下标、一般 for 循环、局部浮点累加器；索引类型与数值类型分离。
- P：定义 i/j 的 GPU 任务映射和唯一写入者；若并行 k，另需R 归约。

性能 / 可选扩展

CPU 内层 i 连续不等于 GPU 地址连续；A/C 按行存储时有 stride。

已有能力与缺口

当前已有标量乘加；此函数的数组循环尚未被 GPU walker 后端支持。

05 loop_kij

需要 GPU 扩展

逐 K 的外积

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- 每个 k 对 C 加上 A[:,k] 与 B[k,:] 的外积。
- 算术量 $O(MNK)$ ，结果 $O(MN)$ ；不需复制乘数对或显式构图。
- 最小可在一个输出线程内串行 k；并行 k 必须归约或原子。

计算示意（不是新增 Jac API）

```
for k in range(K):
    for i in range(M):
        for j in range(N):
            C[i,j] += A[i,k] * B[k,j]
```

HOST 边界

形状检查、设备分配和最终二维列表封装可留在 CPU。

Jac On GPU：最低需要

- D：共享只读 A/B 设备缓冲区；独立可写 C，支持 shape / stride。
- I：整数动态下标、一般 for 循环、局部浮点累加器；索引类型与数值类型分离。
- P：定义 i/j 的 GPU 任务映射和唯一写入者；若并行 k，另需 R 归约。

性能 / 可选扩展

若每个 k 都启动一个 kernel，会重复读写 C 并增加启动成本。

已有能力与缺口

当前已有标量乘加；此函数的数组循环尚未被 GPU walker 后端支持。

06 loop_kji

需要 GPU 扩展

交换外积的遍历方向

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- 同样逐 k 累积外积，但先列后行；CPU 内层 i 访问跨行。
- 算术量 $O(MNK)$ ，结果 $O(MN)$ ；不需复制乘数对或显式构图。
- 切勿直接并行所有 k 后共享写 C ；先定义结果所有权。

计算示意（不是新增 Jac API）

```
for k in range(K):
    for j in range(N):
        for i in range(M):
            C[i,j] += A[i,k] * B[k,j]
```

HOST 边界

形状检查、设备分配和最终二维列表封装可留在 CPU。

Jac On GPU：最低需要

- D：共享只读 A/B 设备缓冲区；独立可写 C，支持 shape / stride。
- I：整数动态下标、一般 for 循环、局部浮点累加器；索引类型与数值类型分离。
- P：定义 i/j 的 GPU 任务映射和唯一写入者；若并行 k ，另需 R 归约。

性能 / 可选扩展

可重排 GPU 任务/物理布局；不能仅凭循环顺序宣称合并访存。

已有能力与缺口

当前已有标量乘加；此函数的数组循环尚未被 GPU walker 后端支持。

07 transposed_dot

需要 GPU 扩展

先转置 B，再做行点积

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出

如何计算 / 数据表示

- 显式生成 $BT[j,k]$ ；A 的行与 BT 的行在各自的 k 方向连续。
- 每个输出单元归约一对连续向量；额外存储 $O(KN)$ 。
- 转置有准备成本；同一 BT 可被多个输出行重复使用。

计算示意（不是新增 Jac API）

```
BT = transpose(B)
for (i,j) in output_cells:
    C[i,j] = sum_k(A[i,k] * BT[j,k])
```

HOST 边界

CPU 可负责一次性转置；零拷贝 view 与物理转置应区分。

Jac On GPU：最低需要

- D/I：设备二维视图、动态索引和局部 K 循环。
- 转置可在 CPU 完成后上传；若保留 GPU 驻留，需转置 kernel 或带 stride 的视图。
- P：一线程一输出；若一 warp 协作点积，需 R 的 lane 归约。

性能 / 可选扩展

线程内 k 连续与 warp 间地址连续是两件事；沿 k 协作时该布局更自然。

已有能力与缺口

标量算术已具备；数组读取和独立点积调度缺失。

08 comprehension

需要 GPU 扩展

用推导式描述输出与归约

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段

如何计算 / 数据表示

- 二维列表推导式枚举输出，sum 对 K 维产品归约。
- 计算语义与逐输出点积一致；变化在 Jac 表达和 lowering。
- GPU 不必创建 Python generator 或大量临时对象。

计算示意（不是新增 Jac API）

```
[[sum(A[i,k] * B[k,j] for k in range(K))
  for j in range(N)]
 for i in range(M)]
```

HOST 边界

二维 Python 列表仅用于输入/输出边界；设备端可使用扁平缓冲区。

Jac On GPU：最低需要

- D/P：推导式结果降为预分配设备数组，并映射输出单元。
- I：捕获循环变量和类型；将 generator / sum 展开为设备循环或归约。
- R：明确求和顺序；并行 sum 需归约实现，串行循环可满足最低要求。

性能 / 可选扩展

可识别为 GEMM 或融合的点积 kernel；这是编译选择，而非语法自动带来的性能。

已有能力与缺口

普通 Jac CPU 已支持推导式；GPU walker 表达式选择器尚不支持。

09 flat_buffers

需要 GPU 扩展

行优先的一维连续缓冲区

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- A/B/C 使用 `array("d")` ; 地址分别由 $i*K+k$ 、 $k*N+j$ 、 $i*N+j$ 计算。
- 消除设备计算对 list-of-lists 对象结构的依赖。
- 仍是 ikj 计算，改变的是存储与寻址。

计算示意（不是新增 Jac API）

```
av = flatten(A); bv = flatten(B)
value = av[i*K+k]
cv[i*N+j] += value * bv[k*N+j]
```

HOST 边界

扁平化、上传和 C 列表重建留在 host ; 允许设备缓冲区跨调用复用。

Jac On GPU : 最低需要

- D : Jac 的固定 dtype 设备 buffer、指针/句柄 ABI、长度与所有权。
- I : 整数乘加寻址、动态 load/store、边界策略 ; 不能直接上传 Python array 对象指针。
- P : 设备 launch 参数与 C 的写入划分 ; 当前私有标量输出ABI 需扩展。

性能 / 可选扩展

最直接的 dense GPU 起点 ; 相邻 j 线程读取 B、写 C 的连续元素。

已有能力与缺口

SoA arena 分配经验可复用 ; 通用 indexed buffer 与 store 尚缺。

10 blocked

需要 GPU 扩展

M/N/K 三维分块

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- 外层枚举 $i_0/j_0/k_0$; 内部按 $i/k/j$ 累加对应输出块。
- min 限制尾块 , 支持不是 tile 整数倍的矩阵。
- CPU 分块本身并不意味着 GPU 已有线程块协作。

计算示意 (不是新增 Jac API)

```
for (i0,j0,k0) in tiles:
    for (i,k,j) in clipped_tile:
        C[i,j] += A[i,k] * B[k,j]
```

HOST 边界

tile 参数与调度配置在 CPU ; 设备内只执行边界已定义的任务。

Jac On GPU : 最低需要

- D/I : 带偏移的 tile view、嵌套循环、min / 边界掩码与整数索引。
- P : 输出 tile 的独立所有权 ; 同一 C tile 的 k_0 阶段需有序累加。
- 最低可单线程算一个 tile ; 高效实现另需 H , 但 H 不是正确性的前提。

性能 / 可选扩展

H : 协作加载 A/B 到 shared memory、barrier、线程寄存器 tile ; 之后再优化流水线。

已有能力与缺口

标量状态可复用 ; 当前不能读取数组子块或写多个 C 元素。

11 packed_tiles

需要 GPU 扩展

把输入子块打包成连续数组

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- 每个阶段复制 A/B 子块到 at / bt，再在块内计算。
- 临时存储 $O(T^2)$ ，换取规则地址和块内复用。
- 本实现会在不同输出 tile 中重复打包；GPU 需评估复制成本。

计算示意（不是新增 Jac API）

```
at = pack(A[i0:i0+T, k0:k0+T])
bt = pack(B[k0:k0+T, j0:j0+T])
C_tile += at @ bt
```

HOST 边界

pack 可先在 CPU 或独立 GPU kernel 完成；两者应分别计入成本。

Jac On GPU：最低需要

- D/I：连续 tile 缓冲区、带边界的复制与块内 indexed load/store。
- 临时空间可由 host 预分配并传入；不必首先实现通用 device malloc。
- P：任务局部 workspace 生命周期；协作共享同一 tile 时需 H 的 shared memory / 同步。

性能 / 可选扩展

采用协作加载、padding/swizzle 可优化访存；局部数组不保证自动进入 shared memory。

已有能力与缺口

已有 arena 管理可作起点；数组临时量和地址空间语义尚缺。

12 register_2x2

需要 GPU 扩展

一个工作项持有四个累加器

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- 读取 a0/a1、b0/b1，更新 c00/c01/c10/c11。
- 每个 K 步复用四个输入，计算四个输出；奇数 M/N 使用边界判断。
- “寄存器”描述目标映射；当前 CPU 实现只是四个局部变量。

计算示意（不是新增 Jac API）

```
c00 += a0*b0; c01 += a0*b1
c10 += a1*b0; c11 += a1*b1
store_2x2_with_tail_mask()
```

HOST 边界

输入保持共享矩阵；输出坐标与尾部尺寸作为 launch metadata。

Jac On GPU：最低需要

- D/I：四路 indexed load、一般 K 循环、边界检查、四路 output store。
- P：一个线程或 walker 拥有 2×2 输出；不同任务互不覆盖。
- 标量 SSA/寄存器分配可承接累加器；不要求 Tensor Core。

性能 / 可选扩展

在更大的 cooperative tile 内组合线程小块；须平衡寄存器占用和并发。

已有能力与缺口

改写为多字段乘数链可复用当前标量状态，但会重新引入 O(MNK) 构图。

13 split_k

需要 GPU 扩展

K 分段计算，再合并部分结果

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- N 数值语义

如何计算 / 数据表示

- 按 chunk 切分 K；每段生成 partial[M,N]，然后加到 C。
- 仓库版本串行计算各段；GPU 并行 split-K 是下一层调度选择。
- 长 K、输出工作项较少时，分段可提供更多可并行任务。

计算示意（不是新增 Jac API）

```
P[s,i,j] = sum_k_in_chunk(s)(A[i,k]*B[k,j])
C[i,j] = sum_s(P[s,i,j])
```

HOST 边界

chunk 与 workspace 分配在 host；无需 device-side kernel launch。

Jac On GPU：最低需要

- D/I/P：段边界、局部 K 循环；最低可一个线程串行执行各段。
- 若并行段：R 要求 partial workspace 与后续归约 kernel / 有序阶段依赖。
- 替代方案是原子累加，但浮点顺序不同；应声明 N 的精度与确定性契约。

性能 / 可选扩展

并行 split-K 可能增加 workspace 和读写；只在需要更多并发时评估。

已有能力与缺口

标量分段累加可复用；跨任务结果合并与 kernel pipeline 尚缺。

14 pairwise

需要 GPU 扩展

二叉树式求和

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- N 数值语义

如何计算 / 数据表示

- 先生成 K 个产品，pair_sum 递归把区间分成两半。
- 树形求和改变舍入路径；通常比长串行累加更有利于误差控制。
- 代码的递归与 GPU reduction tree 可以用不同执行形式表达。

计算示意（不是新增 Jac API）

```
p[k] = A[i,k] * B[k,j]
sum(l,r) = sum(l,mid) + sum(mid,r)
```

HOST 边界

树规划/临时空间可在 host；CPU wrapper 的递归不用全部搬到设备。

Jac On GPU：最低需要

- D/I：产品读取；R：区间树/迭代分层归约与奇数长度处理。
- 保留精确树结构时需支持相同分组；warp shuffle 的另一棵树不保证逐位相等。
- device recursion 不是必需：可展开、用显式栈，或在 host 安排层次 kernel。

性能 / 可选扩展

并行归约可用 shuffle / shared memory；产品数组可融合生成，减少中间写回。

已有能力与缺口

当前只有串行 walker 私有累加；无 tree reduction 或一般 helper 调用。

15 compensated

需要 GPU 扩展

Kahan 补偿求和

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

N 数值语义

如何计算 / 数据表示

- 每个输出保留 total 与 correction；修正每一步产品累加的舍入损失。
- 时间仍 $O(MNK)$ ，每个输出只需常数个标量状态。
- 顺序依赖强，不能用普通并行 sum 直接替代其数值语义。

计算示意（不是新增 Jac API）

```
y = A[i,k]*B[k,j] - correction
t = total + y
correction = (t-total) - y
total = t
```

HOST 边界

两条路线：共享矩阵 + 新数组后端，或已有链后端 + 昂贵的CPU 构图。

Jac On GPU：最低需要

- 保留数组版本：D/I/P 的 indexed load、K 循环、独立输出任务。
- N：保持关键减法/括号，不启用会消除 correction 的 reassociation。
- 可把 y/t/correction 改成 walker 私有 float 字段，再沿产品链执行已有标量操作。

性能 / 可选扩展

优先满足精度语义；并行 Kahan 需定义局部补偿与合并算法，不能默认换普通归约。

已有能力与缺口

链式改写可复用已有多字段 float 状态；原 dense 函数并非当前GPU 可执行。

16 accurate_sum

需要 GPU 扩展

标准库 math.fsum 的高精度求和

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- N 数值语义

如何计算 / 数据表示

- 每个输出把产品流交给 math.fsum。
- fsum 使用多个部分和跟踪精度；语义比一个普通sum 更强。
- 产品本身仍按浮点乘法产生；它不是任意精度 matmul。

计算示意（不是新增 Jac API）

```
C[i,j] = math.fsum(  
    A[i,k] * B[k,j] for k in range(K))
```

HOST 边界

shape 检查和缓冲区分配留 host；求和本体必须在 GPU，才能算GPU 路径。

Jac On GPU：最低需要

- D/I/P：设备产品流和输出映射。
- N/R：提供 fsum 兼容的设备算法/内建，含部分和存储与指定的浮点行为。
- 当前 Python math.fsum 不能在 PTX 内直接调用；替换为 Kahan 需明确语义变化。

性能 / 可选扩展

先建立误差与顺序验证，再决定串行多部分和、并行补偿或更强累加器。

已有能力与缺口

普通乘加已有；fsum helper、临时数组与其数值语义均未实现。

17 winograd_pairs

需要 GPU 扩展

用成对恒等式减少主循环乘法

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- N 数值语义

如何计算 / 数据表示

- 预计算 A 行因子 r_i 与 B 列因子 q_j 。
- 每对 K 使用 $(a_0+b_1)*(a_1+b_0)$ ，再扣除两个因子。
- 奇数 K 补上最后一项；这是经典成对公式，与 Strassen 变体不同。

计算示意（不是新增 Jac API）

```
r[i] = sum_p(A[i,2p]*A[i,2p+1])
q[j] = sum_p(B[2p,j]*B[2p+1,j])
C = -r[i]-q[j] + sum_p((a0+b1)*(a1+b0))
```

HOST 边界

因子可先在 host 计算；全 GPU 路径需把预处理也纳入设备阶段。

Jac On GPU：最低需要

- D/I：成对 indexed load、整数步长与奇数 K 的边界条件。
- R/P：行因子、列因子与输出计算的阶段依赖；可用多个 kernel 或同任务计算。
- N：额外加减带来抵消风险；GPU 验证不能仅要求乘法次数减少。

性能 / 可选扩展

乘法减少不保证更快；额外预处理、读写和加法都要计入。

已有能力与缺口

成对链 + 预先给定因子可复用标量算术；共享数组路径仍需扩展。

18 threaded_rows

需要 GPU 扩展

CPU 线程池式行任务

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

R 归约 / 多阶段

如何计算 / 数据表示

- 线程池提交每个 A 行与所有 BT 列的点积任务。
- 4 个 CPU worker 与 GPU 的 thread/block/warp 是不同调度模型。
- 输出行彼此独立，适合显式描述并行任务。

计算示意（不是新增 Jac API）

```
pool.submit(row_product, A[i,:], BT)
C[i,:] = future.result()
```

HOST 边界

线程池可留作提交层，但不要将 Python 线程执行误称为设备计算。

Jac On GPU：最低需要

- P：将独立行任务映射成 GPU grid / 行块；CPU Future 不能作为设备同步原语。
- D/I：共享只读数组、行内输出、K 循环；协作点积另需 R。
- 异步 host 接口需 stream/event 完成语义；同步 launch 已可满足最低要求。

性能 / 可选扩展

一行一个线程可能并发不足；可把列分给多个线程或改用 tile。

已有能力与缺口

批量独立 walker 的调度思路已有；ThreadPoolExecutor 本体只在 CPU。

19 overloaded_operator

需要 GPU 扩展

Jac 对象重载 a @ b

D 设备数组 / view

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- Matrix 对象持有 data ; __matmul__ 调用 loop_ikj。
- 它是运算符与对象接口路径，底层算法没有改变。
- GPU 关键是让对象携带设备数据，并确定 @ 的目标实现。

计算示意（不是新增 Jac API）

```
obj Matrix: data
def __matmul__(other):
    return Matrix(loop_ikj(data, other.data))
```

HOST 边界

重载解析可在编译时或 host 进行；无需把动态 Python 对象协议搬进PTX。

Jac On GPU : 最低需要

- D : Matrix 的设备 tensor/view 表示；对象不必原样复制到GPU。
- I/P : 静态类型化的重载分派、方法内联或设备函数调用，以及 dense kernel lowering。
- 也可在 host 将 @ 分派到 GPU 库；但这属于 L 路线，而非现有函数自动支持。

性能 / 可选扩展

同一 @ 可选择 scalar、tiled 或库 kernel；选择依据应含 dtype / shape / layout。

已有能力与缺口

Jac CPU 已支持 __matmul__；GPU walker 仅允许字段和少量标量表达式。

20 recursive_blocks

需要 GPU 扩展

递归拆分最大维度

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段

如何计算 / 数据表示

- 在 M、N、K 中选择最大维度，递归对半拆分。
- 用 i/j/k 区间表示子矩阵，避免复制象限；叶子调用小循环。
- 拆 M/N 的结果互不相交；拆 K 的子问题累加到同一输出。

计算示意（不是新增 Jac API）

```
if max(M,N,K) <= 2: small_gemm()  
else: split_largest_dimension()  
      recurse(left); recurse(right)
```

HOST 边界

推荐 host 规划 + GPU 叶子；device recursion 不是这个算法的必要特性。

Jac On GPU：最低需要

- D/I：带偏移与 stride 的矩阵 view、小 GEMM 的循环/写入。
- P：独立 M/N 子任务可并行；K 分裂需串行顺序或 R 的 partial reduction。
- 递归可在 CPU 展开为工作清单；若照搬设备递归才需额外调用栈支持。

性能 / 可选扩展

避免大量微小 kernel；合并叶子任务并选择 cutoff / tile 尺寸。

已有能力与缺口

标量叶子算术已有；递归调用、视图和输出写入尚无完整 GPU lowering。

21 strassen

需要 GPU 扩展

七次子乘法的递归分治

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- N 数值语义

如何计算 / 数据表示

- 2×2 象限乘法从 8 次子乘法改成 7 次，再加减重组。
- 先把矩阵补为覆盖 M/K/N 的二次幂方阵；最终裁剪。
- 方阵递推约 $O(S^{\log_2(7)})$ ，但 padding、复制和 workspace 有代价。

计算示意（不是新增 Jac API）

```
P1..P7 = gemm(combinations_of_A_B)
C11 = P1+P4-P5+P7
C12 = P3+P5; C21 = P2+P4
C22 = P1-P2+P3+P6
```

HOST 边界

递归规划留 host；中间矩阵驻留设备，避免每层往返拷贝。

Jac On GPU：最低需要

- D/I：象限 view / 复制、矩阵加减、padding / crop 和叶子GEMM。
- R/P：七个子任务的依赖图、临时缓冲区生命周期、输出组合kernel。
- N：增加加减和抵消；需 cutoff 与误差策略；可 host 递归调度，无需动态设备 launch。

性能 / 可选扩展

大子问题用 tiled GEMM 或 GPU 库；小矩阵/细长矩阵不保证获益。

已有能力与缺口

现有 scalar walker 不能直接表达矩阵临时量与跨 kernel 的依赖。

22 strassen_winograd

需要 GPU 扩展

Strassen 的 Winograd 变体

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- N 数值语义

如何计算 / 数据表示

- 保持 7 次子乘法，使用另一组 S/T 中间量与重组关系。
- 标准这一变体每层 15 次矩阵加减；本实现含象限复制与padding。
- 不要与 winograd_pairs 的 K 成对公式混为一谈。

计算示意（不是新增 Jac API）

```
S1..S8 = combinations_of_quadrants
M1..M7 = recursive_gemm(...)
T1..T2 = intermediate_sums
C11..C22 = recombine(M,T)
```

HOST 边界

host 安排 S/M/T 阶段；设备执行矩阵算子，workspace 可预先规划。

Jac On GPU：最低需要

- D/I：象限、加减、补零/裁剪、叶子 GEMM，与 Strassen 共享基础能力。
- R/P：使用本变体的依赖图和 workspace 复用，而不是照抄另一种重组顺序。
- N：严格保护公式与符号；验证精度和 cutoff，递归可由host 组织。

性能 / 可选扩展

减少加减数量仍需考虑带宽、临时存储和任务粒度；不能由名字推断性能。

已有能力与缺口

当前缺矩阵临时量与多阶段调度；七乘法公式已有 CPU 验证。

23 csr_dense

需要 GPU 扩展

CSR 稀疏 A × 稠密 B

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出

如何计算 / 数据表示

- 把 A 的非零值打包成 offsets / indices / values。
- 每行遍历非零项，再对 B[k,:] 做缩放累加；C 为稠密矩阵。
- 算术约 $O(\text{nnz}(A) \cdot N)$ ；稠密输入转 CSR 仍需扫描 $O(MK)$ 。

计算示意（不是新增 Jac API）

```
for p in offsets[i]:offsets[i+1]:
    k = indices[p]
    C[i,:] += values[p] * B[k,:]
```

HOST 边界

CSR 构造可留 CPU，之后上传一次；GPU 输入已是 CSR 时应避免重打包。

Jac On GPU：最低需要

- D：通用 CSR 索引/值列及共享 B/C；图 CSR 邻接并不是自动可用的稀疏数值矩阵。
- I：变长区间循环、indirect load B[k,j]、整数索引和浮点值共同使用。
- P：输出行/列唯一所有权；并行同一行的 p 时另需归约。

性能 / 可选扩展

H/R 可用于行内协作与负载均衡；L 可选择 cuSPARSE SpMM 作为替代后端。

已有能力与缺口

已有 CSR traversal 管理 offsets/targets；没有此算法的数组寻址与数值 edge/value 语义。

24 csr_csc_merge

需要 GPU 扩展

稀疏行与稀疏列双指针合并

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出

如何计算 / 数据表示

- A 行和 B 列分别存有序 (k,value) 列表。
- 两个指针比较 k；相等才相乘，否则前移较小的一方。
- 每个输出独立；仍枚举 MN 个输出，收益依赖稀疏结构。

计算示意（不是新增 Jac API）

```
while p < nnz_row and q < nnz_col:
    if ar[p].k == bc[q].k: accumulate; p++; q++
    elif ar[p].k < bc[q].k: p++
    else: q++
```

HOST 边界

排序与格式转换可留 host；设备 kernel 假设区间中的 k 已有序。

Jac On GPU：最低需要

- D：CSR/CSC 风格 ragged buffers；元组可扁平化为 index/value 列。
- I：一般 while、多路分支、两个独立动态游标、indirect loads。
- P：每个 (i,j) 独占 total；无需全局原子，可先采用一线程一输出。

性能 / 可选扩展

长短行差异会导致 warp 分歧；可分桶或使用 cooperative intersection。

已有能力与缺口

当前支持 if，但不支持一般 while 或对多个数组游标的索引读取。

25 coo_scatter

需要 GPU 扩展

COO 乘积向结果散射累加

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- N 数值语义

如何计算 / 数据表示

- A 用 (i,k,value) 项列表 , B 按 k 保存非零 (j,value)。
- 每一对非零乘法贡献散射到 C[i,j]。
- 工作量取决于结构连接数 ; 多个贡献可能命中同一输出。

计算示意 (不是新增 Jac API)

```
for (i,k,a) in A_coo:
  for (j,b) in B_sparse_row[k]:
    C[i,j] += a*b
```

HOST 边界

host 可负责分组 ; 若要求设备生成稀疏结构 , 另需 scan / compaction。

Jac On GPU : 最低需要

- D/I : COO 字段列、B 的 ragged rows、indexed scatter writes。
- P/R : 若不同贡献并行 , 需 float64 原子累加或先分组再归约 ; 串行所有者可避免原子。
- N : 原子累加的顺序可能变化 ; 固定结果顺序需有序分段归约。

性能 / 可选扩展

按输出行/位置分组以降低冲突 ; GPU 全量并行并不是唯一路线。

已有能力与缺口

现有 runtime 只允许私有 walker 状态与独立输出 , 没有共享 C 写语义。

26 product_chains

已有可运行 GPU 对应版

一个输出 walker 走一条乘数链

G 图语义扩展

如何计算 / 数据表示

- MNK 个 Product 节点，每个保存 left/right 两个 float。
- MN 条链，各长 K；MN 个 walker 各自累加一个输出。
- 图载荷与构图 $O(MNK)$ ，复制输入；正确但准备成本明显。

计算示意（不是新增 Jac API）

```
self.total += here.left * here.right;  
visit [->:NextProduct:->];
```

HOST 边界

CPU 创建图/对象并打包上传；GPU 线程完成整条链；最终结果回写host。

Jac On GPU：最低需要

- 已有：scalar float 字段、私有累计状态、单一类型 tail visit 与 batch launch。
- 此函数的 CPU spawn 不会自动上 GPU；使用 GpuWalkerRuntime 批量提交 starts/walkers。
- 无需新算法语义即可运行独立 GPU 示例；减少构图需 D 或新的隐式拓扑能力。

性能 / 可选扩展

按 K 层、walker 顺序排节点可改善同一步访问；仍有重复数据和 CPU 打包开销。

已有能力与缺口

独立 GPU 版本 jac/examples/gpu/matmul.jac 已在会话中实机验证。

27 dual_cursors

需要 GPU 扩展

共享标量节点，两个同步游标

G 图语义扩展

P 任务映射 / 输出

如何计算 / 数据表示

- A 行链 + B 列链，共 $MK+KN$ 个标量节点。
- 主位置 here 走 A 行；self.rhs 指向 B 列的当前节点。
- MN 个 walker 不复制乘数；边数 $M(K-1)+N(K-1)$ 。

计算示意（不是新增 Jac API）

```
self.total += here.value * self.rhs.value;
following = [self.rhs->:ColumnNext:->];
self.rhs = following[0];
visit [->:RowNext:->];
```

HOST 边界

最低路线仍可 CPU 构建 $O(MK+KN)$ 图；隐式 row/column view 可进一步省对象。

Jac On GPU：最低需要

- G：walker 中的 device node handle，远端字段读取 self.rhs.value。
- G：从任意 receiver 查询另一边类型、返回/索引 query 结果，并更新第二游标。
- packer：同时收集 A/B 根与拓扑，重映射 handles；现有单一 tail-visit 图打包不够。

性能 / 可选扩展

节点少不保证 coalescing；B 列链布局与相邻 walker 的列分配必须联合设计。

已有能力与缺口

链遍历和标量乘加已有；节点引用不是可展开的普通值对象字段。

28 row_nodes

需要 GPU 扩展

行数组节点，一 walker 输出整行

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- G 图语义扩展

如何计算 / 数据表示

- M 个 DenseRow 节点，每个持有 K 长度数组；无边。
- 每个 walker 共享 BT，与所有列做点积，输出 N 个数。
- 逻辑行节点并不要求设备上逐行独立 malloc；可映射同一二维 buffer。

计算示意（不是新增 Jac API）

```
node DenseRow: values[K]
walker RowDot: columns[N,K], output[N]
output = [dot(values,col) for col in columns]
```

HOST 边界

CPU 可以创建 M 个任务；数组合并上传并共享，避免每 walker 再拷贝BT

Jac On GPU：最低需要

- D：数组字段 / tensor view，共享 BT 句柄与有界输出向量。
- I/R：zip、推导式与 sum 的 lowering；可先展开成局部循环。
- G/P：支持无 tail visit 的 compute entry；一行任务到线程 /warp/块的映射。

性能 / 可选扩展

让列方向协作，避免一线程串行算整行；row-level 语义与物理 tile 可以分开。

已有能力与缺口

现有 fixed obj flattening 只覆盖标量叶子，不能代替 list 字段支持。

29 output_cell_tasks

需要 GPU 扩展

坐标任务节点，共享 A/B

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- G 图语义扩展

如何计算 / 数据表示

- MN 个 CellTask 节点只保存 row/col，无边。
- walker 用共享矩阵、K 循环计算一个 C[i,j]。
- 不需要乘数图，但若照搬 CPU 版本仍要创建 MN 个任务对象。

计算示意（不是新增 Jac API）

```
node CellTask: row:int, col:int
  for k in range(K):
    total += A[here.row,k] * B[k,here.col]
```

HOST 边界

最低可 CPU 构建坐标节点；更完整方案是 shape 定义逻辑任务，GPU 生成坐标。

Jac On GPU：最低需要

- D/I：walker 携带设备数组句柄、动态下标和 K 循环。
- G：解除必须 tail visit 的限制；整数坐标与 float 累加状态跨 schema 混用。
- P：任务坐标可由 lane 生成；隐式任务空间能避免 CPU 创建MN 个节点。

性能 / 可选扩展

相邻任务沿 j 排列便于 B/C 连续访存；之后可以扩展 tile 协作。

已有能力与缺口

已有私有 total 与 batch 思路；数组句柄、计算型 entry 和 int/float 混合限制尚需处理。

30 output_tile_tasks

需要 GPU 扩展

一个任务代表一个输出 tile

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- G 图语义扩展

如何计算 / 数据表示

- 任务数 $\text{ceil}(M/T) * \text{ceil}(N/T)$ ，每节点只存 tile 起点。
- walker 串行 K/i/j 循环，保留局部输出数组；没有真实协作线程。
- A/B 共享，输出块唯一归属；尾块按实际 im/jn 裁剪。

计算示意（不是新增 Jac API）

```
node TileTask: row, col
  output[im,jn] = 0
  for k,i,j: output[i,j] += A[row+i,k]*B[k,col+j]
```

HOST 边界

不必由 CPU 创建 tile 图：shape + tile 尺寸可以定义虚拟任务空间。

Jac On GPU：最低需要

- D/I：tile view、边界掩码、有界私有数组或寄存器片段、输出store。
- G/P：无 tail visit 的计算型 entry，混合 metadata/value 类型与隐式 tile task 生成。
- 最低一线程/任务已能跑；要映射成 CUDA block 需新增 H 的协作与同步契约。

性能 / 可选扩展

H：block 协作加载、shared memory、barrier、分布式寄存器累加；Tensor Core 是后续可选项。

已有能力与缺口

任务/私有状态概念可复用；“一个 walker”目前不自动等于一个线程块。

31 bipartite_join

需要 GPU 扩展

共享输入节点，通过邻居做乘积连接

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- G 图语义扩展

如何计算 / 数据表示

- LeftValue 共 MK，RightValue 共 KN，节点复用输入。
- 同 K 连接 MKN 条 SameK 边，另加 A 行链。
- M 个 walker，每次访问 A 元素时查所有匹配 B 并更新输出行。

```
计算示意（不是新增 Jac API）
for other in [->:SameK:->]:
    output[other.column] += here.value * other.value;
visit [->:NextLeft:->];
```

HOST 边界

若照搬需 CPU 构建 MKN 边；迁移为索引连接会改变物理表示而保留关系语义。

Jac On GPU：最低需要

- G：多边关系、普通邻居查询 / 结果循环、远端属性读取、异构节点表示。
- D/I：walker 私有 output[N]，动态列写入；RightValue 的 float + int 混合字段。
- P：同一行的唯一写入者可免原子；并行邻居贡献时另需归约/写冲突规则。

性能 / 可选扩展

节点复用但边仍 $O(MNK)$ ；可用按 k 的隐式连接代替完全显式 SameK 图。

已有能力与缺口

已有 tail visit；当前选择器不能表达查询循环、第二边关系与异构访问。

32 edge_payloads

需要 GPU 扩展

乘数存在边上，walker 归约边属性

G 图语义扩展

I 索引 / 控制流

P 任务映射 / 输出

如何计算 / 数据表示

- MN 个输出任务节点，K 个 ReductionIndex 节点。
- 每个输出连 K 条 Factor 边，边持有 left/right float。
- MN 个 walker 查询边并累加；MNK 边仍显式复制输入。

计算示意（不是新增 Jac API）

```
for factor in [edge ->:Factor:->]:
    self.total += factor.left * factor.right;
```

HOST 边界

最低 host 构建并打包带 payload 的边；若去重应改为共享输入句柄或view。

Jac On GPU：最低需要

- G：带属性 edge 的 SoA 列、edge query、edge handle 与属性load。
- I：按 CSR 区间迭代边；G/P：无 tail visit 的 entry 和输出任务batch。
- 节点索引字段为 int、累计为 float；需要按字段/角色分离 dtype。

性能 / 可选扩展

同一 K 层的边列可按输出任务交错；保存边属性并不能自动解决MNK 复制。

已有能力与缺口

现有 CSR targets 只描述连向哪里；并未提供 Factor 的字段列或 edge query lowering。

33 numpy_operator

需要 GPU 库 / intrinsic

NumPy 数组的 @ 运算符

- D 设备数组 / view
- P 任务映射 / 输出
- L GPU 库互操作

如何计算 / 数据表示

- 二维 dense GEMM ; operator 路径需解析数组类型对应的GPU 实现。
- 当前代码经 Jac 的 Python 互操作调用 CPU NumPy ; 并未在 GPU 上运行。
- GPU 路线可用编译 intrinsic , 或 host 调用 GPU 库 , 二者应明确区分。

计算示意 (不是新增 Jac API)

```
av @ bv
```

HOST 边界

host 负责库调用和形状分派 ; 矩阵乘法由 GPU 库 kernel 执行。

Jac On GPU : 最低需要

- D/L : Jac 设备 tensor ABI、dtype/layout、显存与 stream/context 生命周期 ; 接入 GPU GEMM。
- 类型化 operator 分派 ; 不要把 numpy.ndarray 原样当作设备buffer。
- P/L : host launch 后同步/错误检查 ; 普通 import / NumPy 调用不能在 PTX 内直接执行。

性能 / 可选扩展

可复用成熟 GEMM 后端 ; 输入驻留与 layout 适配通常比保留 CPU API 字面形式更重要。

已有能力与缺口

当前 Python 互操作已有 ; GPU tensor 和 cuBLAS/等价库适配尚未建立。

34 numpy_matmul

需要 GPU 库 / intrinsic

NumPy matmul 接口

- D 设备数组 / view
- P 任务映射 / 输出
- L GPU 库互操作

如何计算 / 数据表示

- 本例是二维 GEMM；完整 NumPy 接口还包括广播 batch，不能默认声称已覆盖。
- 当前代码经 Jac 的 Python 互操作调用 CPU NumPy；并未在 GPU 上运行。
- GPU 路线可用编译 intrinsic，或 host 调用 GPU 库，二者应明确区分。

计算示意（不是新增 Jac API）

```
np.matmul(av, bv)
```

HOST 边界

host 负责库调用和形状分派；矩阵乘法由 GPU 库 kernel 执行。

Jac On GPU：最低需要

- D/L：Jac 设备 tensor ABI、dtype/layout、显存与 stream/context 生命周期；接入 GPU GEMM。
- 识别 matmul 调用；若扩展 batch，需 shape 广播、stride 和 batched GEMM。
- P/L：host launch 后同步/错误检查；普通 import / NumPy 调用不能在 PTX 内直接执行。

性能 / 可选扩展

可复用成熟 GEMM 后端；输入驻留与 layout 适配通常比保留 CPU API 字面形式更重要。

已有能力与缺口

当前 Python 互操作已有；GPU tensor 和 cuBLAS/等价库适配尚未建立。

35 numpy_dot

需要 GPU 库 / intrinsic

NumPy dot 的二维矩阵路径

- D 设备数组 / view
- P 任务映射 / 输出
- L GPU 库互操作

如何计算 / 数据表示

- 本例二维 dot 等于矩阵乘法；其他秩的 dot 语义与 matmul 不完全一样。
- 当前代码经 Jac 的 Python 互操作调用 CPU NumPy；并未在 GPU 上运行。
- GPU 路线可用编译 intrinsic，或 host 调用 GPU 库，二者应明确区分。

计算示意（不是新增 Jac API）

```
np.dot(av, bv)
```

HOST 边界

host 负责库调用和形状分派；矩阵乘法由 GPU 库 kernel 执行。

Jac On GPU：最低需要

- D/L：Jac 设备 tensor ABI、dtype/layout、显存与 stream/context 生命周期；接入 GPU GEMM。
- 按已验证的二维签名专门化；更高秩需要单独 shape/axis 规则。
- P/L：host launch 后同步/错误检查；普通 import / NumPy 调用不能在 PTX 内直接执行。

性能 / 可选扩展

可复用成熟 GEMM 后端；输入驻留与 layout 适配通常比保留 CPU API 字面形式更重要。

已有能力与缺口

当前 Python 互操作已有；GPU tensor 和 cuBLAS/等价库适配尚未建立。

36 numpy_einsum

需要 GPU 库 / intrinsic

Einstein 下标描述收缩

- D 设备数组 / view
- P 任务映射 / 输出
- L GPU 库互操作

如何计算 / 数据表示

- 本表达式按 k 收缩输出 ij，可识别为 GEMM；任意 einsum 需要更通用 planner。
- 当前代码经 Jac 的 Python 互操作调用 CPU NumPy；并未在 GPU 上运行。
- GPU 路线可用编译 intrinsic，或 host 调用 GPU 库，二者应明确区分。

计算示意（不是新增 Jac API）

```
np.einsum("ik,kj->ij", av, bv, optimize=True)
```

HOST 边界

host 负责库调用和形状分派；矩阵乘法由 GPU 库 kernel 执行。

Jac On GPU：最低需要

- D/L：Jac 设备 tensor ABI、dtype/layout、显存与 stream/context 生命周期；接入 GPU GEMM。
- 解析下标、shape/stride 和收缩顺序；当前这一式可直接选GEMM。
- P/L：host launch 后同步/错误检查；普通 import / NumPy 调用不能在 PTX 内直接执行。

性能 / 可选扩展

可复用成熟 GEMM 后端；输入驻留与 layout 适配通常比保留 CPU API 字面形式更重要。

已有能力与缺口

当前 Python 互操作已有；GPU tensor 和 cuBLAS/等价库适配尚未建立。

37 numpy_tensordot

需要 GPU 库 / intrinsic

指定轴的 tensor 收缩

- D 设备数组 / view
- P 任务映射 / 输出
- L GPU 库互操作

如何计算 / 数据表示

- 本例收缩 A 的轴 1 与 B 的轴 0；可以整理成二维 GEMM。
- 当前代码经 Jac 的 Python 互操作调用 CPU NumPy；并未在 GPU 上运行。
- GPU 路线可用编译 intrinsic，或 host 调用 GPU 库，二者应明确区分。

计算示意（不是新增 Jac API）

```
np.tensordot(av, bv, axes=([1],[0]))
```

HOST 边界

host 负责库调用和形状分派；矩阵乘法由 GPU 库 kernel 执行。

Jac On GPU：最低需要

- D/L：Jac 设备 tensor ABI、dtype/layout、显存与 stream/context 生命周期；接入 GPU GEMM。
- 轴配对、reshape/view、必要的 permutation；不能默认任意布局都零拷贝。
- P/L：host launch 后同步/错误检查；普通 import / NumPy 调用不能在 PTX 内直接执行。

性能 / 可选扩展

可复用成熟 GEMM 后端；输入驻留与 layout 适配通常比保留 CPU API 字面形式更重要。

已有能力与缺口

当前 Python 互操作已有；GPU tensor 和 cuBLAS/等价库适配尚未建立。

38 numpy_broadcast

需要 GPU 库 / tensor 扩展

广播生成全部产品，再沿 K 求和

- D 设备数组 / view
- I 索引 / 控制流
- P 任务映射 / 输出
- R 归约 / 多阶段
- L GPU 库互操作

如何计算 / 数据表示

- A reshape 为 (M,K,1) , B reshape 为 (1,K,N)。
- 广播乘法得到 $M \times K \times N$ 中间数组，再沿 axis=1 求和。
- 这是表达路径；若显式物化，存储 $O(MKN)$ ，远大于输入和输出。

计算示意（不是新增 Jac API）

```
P = A.reshape(M,K,1) * B.reshape(1,K,N)
C = sum(P, axis=1)
```

HOST 边界

host 控制 tensor API；全 GPU 流程不应把中间 P 下载到 CPU。

Jac On GPU：最低需要

- D：设备 tensor 广播 / reshape、elementwise multiply 与 axis reduction。
- R/I：K 归约顺序与输出布局；P：中间 workspace 和两阶段调度。
- 可通过 L 的 GPU tensor 库或 native lowering 实现；融合后无需真的创建 MKN 数组。

性能 / 可选扩展

优先融合 broadcast + reduction 或识别为 GEMM；否则中间数组的显存/带宽成本很高。

已有能力与缺口

已有标量乘法，未有广播 tensor 或通用 GPU reduce；当前 NumPy 仍CPU。

39 scipy_blas

需要 GPU 库互操作

SciPy BLAS dgemm 包装

- D 设备数组 / view
- P 任务映射 / 输出
- L GPU 库互操作

如何计算 / 数据表示

- Jac 经 Python 调用 SciPy dgemm(alpha=1.0) , 结果为 dense float64。
- BLAS 是接口与底层实现路线 ; GPU 对应可使用 cuBLAS DGEMM。
- 接口约定含 leading dimension / transpose , 不能忽略 row/column-major 适配。

计算示意 (不是新增 Jac API)

```
C = dgemm(1.0, A_float64, B_float64)
# GPU alternative: cublasDgemm / equivalent
```

HOST 边界

调用在 host , 计算在 GPU ; 这是 Jac GPU 互操作路线 , 不是 walker 逐项执行。

Jac On GPU : 最低需要

- L : host FFI 或 GPU 库 wrapper、可用库探测、handle 和返回状态管理。
- D : 设备指针、float64、lda/ldb ldc、转置/存储序 , 以及可复用C。
- P/L : 与 Jac CUDA context / stream 协调 ; SciPy CPU dgemm 不能自动变成设备函数。

性能 / 可选扩展

可作为 dense GPU 正确性与性能对照 ; 按实际 shape / dtype 选择后端。

已有能力与缺口

现有 ctypes CUDA Driver API 可复用 host FFI 思路 ; cuBLAS 集成尚无。

40 scipy_sparse

需要 GPU 稀疏库互操作

CSR × CSR 后转为稠密输出

D 设备数组 / view

P 任务映射 / 输出

R 归约 / 多阶段

L GPU 库互操作

如何计算 / 数据表示

- SciPy 将 A/B 转 CSR，执行稀疏矩阵乘法，再 toarray。
- GPU 对应是 SpGEMM；与 csr_dense 的 SpMM 是不同操作。
- 结果 nnz 事先未知；最终 dense C 仍需要 MN 空间。

计算示意（不是新增 Jac API）

```
Cs = CSR(A) @ CSR(B)
C = Cs.toarray()
# GPU: SpGEMM + sparse_to_dense
```

HOST 边界

稀疏格式转换可先在 CPU；保证乘法和需要的输出转换在 GPU 或明确边界。

Jac On GPU：最低需要

- D/L：稀疏 tensor descriptor、index/value buffers 和 GPU SpGEMM 接口。
- R/P：符号/容量估计、workspace、结果 nnz 分配、计算阶段和稀疏转稠密。
- 若走 native Jac 而非库：还需 join、去重归约、scan / compaction 等完整稀疏算子。

性能 / 可选扩展

复用 sparsity pattern / descriptor 可减少准备；是否保留 dense 输出应由应用决定。

已有能力与缺口

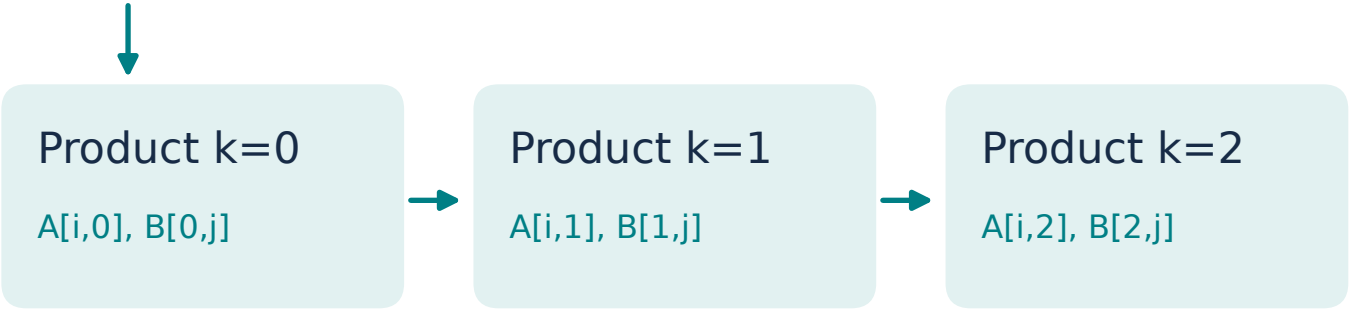
图 CSR 的 traversal 不提供 SpGEMM 的结构生成和数值归约语义。

同一个点积：乘数链与双游标如何不同

示例固定输出 $C[i,j]$ ， $K=3$ ；不同输出的 walker 可以复用原始 A/B 标量节点。

product_chains：复制每一次乘法的输入

ProductSum walker



NextProduct → 逐项累加 total

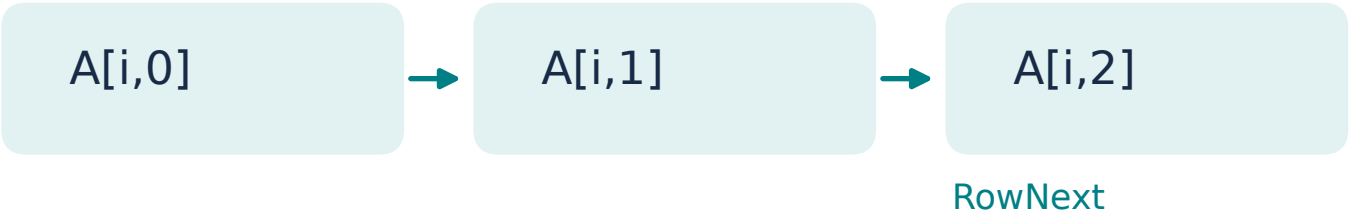
MN 条链 × K 个节点

原始 A/B 数据在多个输出链中重复。

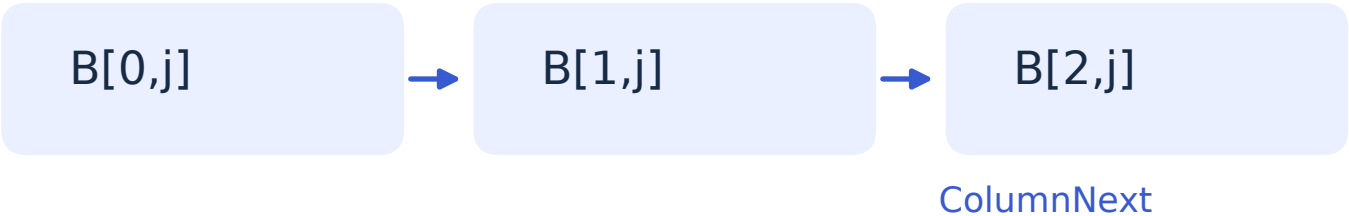
已有 GPU 模型；CPU 构图/上传 $O(MNK)$ 。

dual_cursors：一份 A/B，两条游标

主位置 here：A 的第 i 行



第二句柄 rhs：B 的第 j 列



同一 k 读取两个节点并相乘；分别前移。

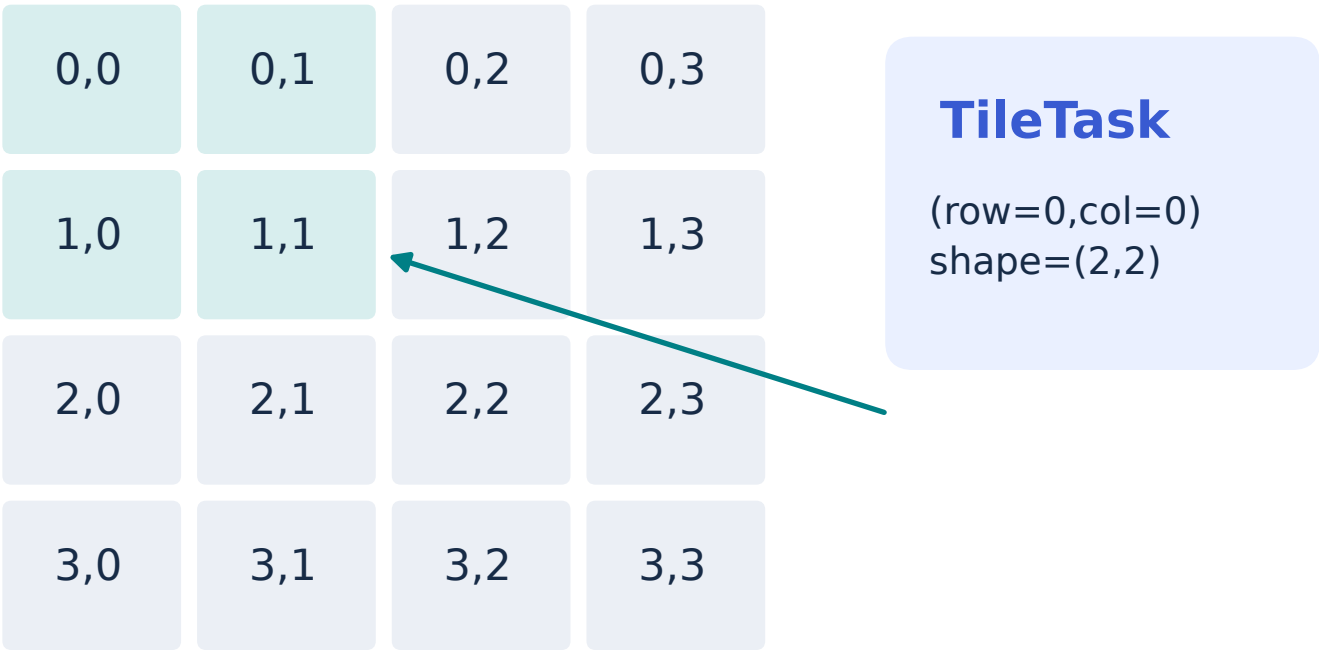
需 node handle + 远端 load + 第二 receiver query；
图大小 $O(MK+KN)$ ，不再为每个输出复制输入。

逻辑关系来自 graphs.jac；箭头是类型边，设备表示还需索引重映射。

输出 **tile** 任务：逻辑 **OSP** 与 **GPU** 协作分别定义

示意设计方向；当前 TileDot 在 CPU 上串行算 tile，并未实现图中的 block 协作。

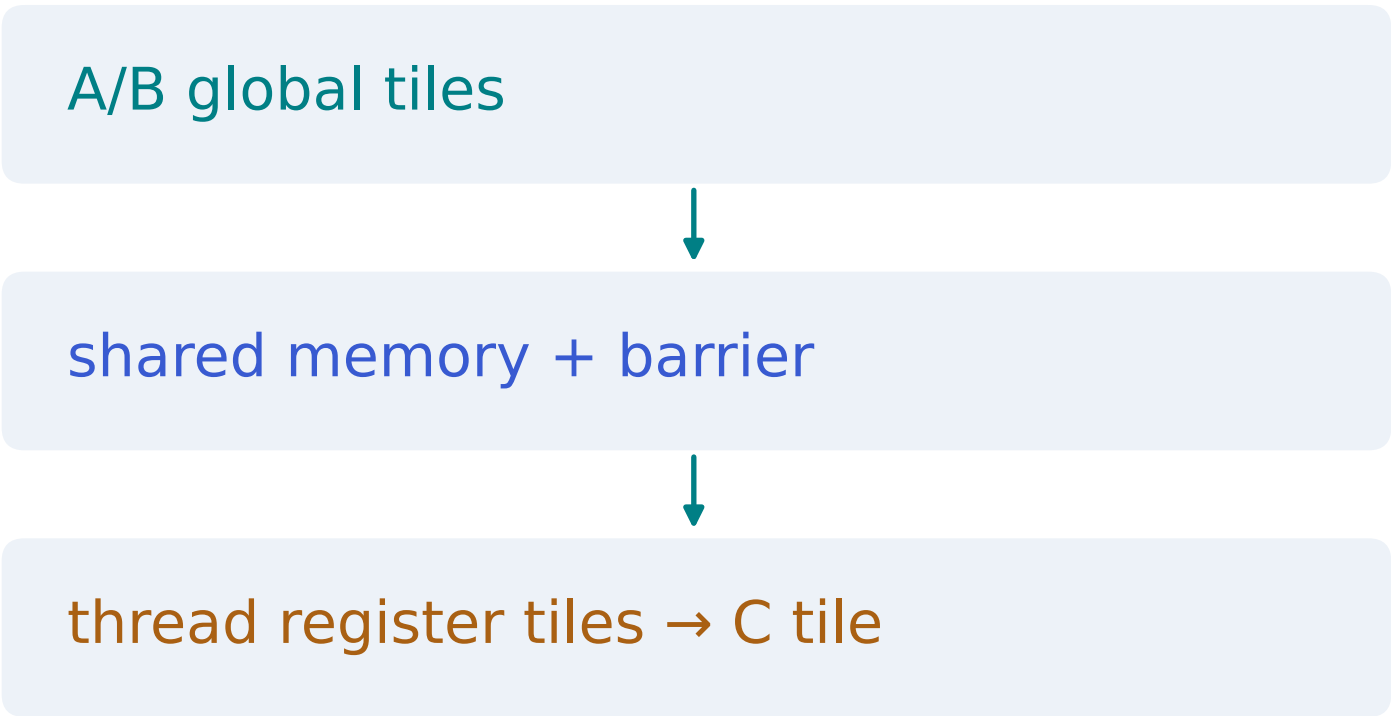
逻辑层：任务只表示输出范围



共享 A/B view + 独占 C 子块；无须显式乘法边。

任务坐标可以由 GPU lane/block 生成。

物理层：高性能扩展 **H**



walker → block 需要明确协作映射；shared / registers 是不同地址空间，不能靠普通 list 自动获得。

右图为建议的 Jac 功能映射；GPU 分层计算依据 NVIDIA CUTLASS Efficient GEMM。

七种 **OSP**：省节点，不一定省边或 **CPU** 准备

数量来自当前 CPU 模型；M×K 与 K×N 输入。G 表示 graph 能力，D/I/P 表示数组与任务能力。

实现	节点数	边数	walker / 结果	主要缺口
product_chains	MNK	MN(K−1)	MN，标量	已有链 GPU；构图大
dual_cursors	MK+KN	(M+N)(K−1)	MN，标量	G：第二句柄/receiver
row_nodes	M	0	M，N 向量	D/R：行 view + 归约
output_cell_tasks	MN	0	MN，标量	D/I/P：共享数组/坐标
output_tile_tasks	ceil(M/T)ceil(N/T)	0	tile 数，T² 输出	D/I/P；H 使 tile 协作
bipartite_join	MK+KN	MKN+M(K−1)	M，N 向量	G/D：query + 行输出
edge_payloads	MN+K	MNK	MN，标量	G：edge payload/query

数组/任务可表达逻辑关系而不物化完整图；双游标复用输入，但仍要定义物理布局与 warp 访问。

功能对照表 1/4：实现 01-13

● 需处理的能力域；○ 常见可选优化。各页区分最低与条件需求；标签不代表当前 GPU 支持。

实现	D	I	P	R	G	H	L	N	状态
01 loop_ijk	●	●	●			○			需扩展/改写
02 loop_ikj	●	●	●			○			需扩展/改写
03 loop_jik	●	●	●			○			需扩展/改写
04 loop_jki	●	●	●			○			需扩展/改写
05 loop_kij	●	●	●			○			需扩展/改写
06 loop_kji	●	●	●			○			需扩展/改写
07 transposed_dot	●	●	●			○			需扩展/改写
08 comprehension	●	●	●	●		○			需扩展/改写
09 flat_buffers	●	●	●			○			需扩展/改写
10 blocked	●	●	●			○			需扩展/改写
11 packed_tiles	●	●	●			○			需扩展/改写
12 register_2x2	●	●	●			○			需扩展/改写
13 split_k	●	●	●	●		○		●	需扩展/改写

D 数据 · I 索引/控制 · P 任务/结果 · R 显式归约/阶段 · G 图语义 · H 协作 · L 库 · N 数值契约
R 不含普通串行 += ；H 与并行 K 的需求依调度选择。L 标注保留 GPU 库路径时的互操作要求。

功能对照表 2/4：实现 14-25

● 需处理的能力域；○ 常见可选优化。各页区分最低与条件需求；标签不代表当前 GPU 支持。

实现	D	I	P	R	G	H	L	N	状态
14 pairwise	●	●	●	●		○		●	需扩展/改写
15 compensated	●	●	●					●	需扩展/改写
16 accurate_sum	●	●	●	●				●	需扩展/改写
17 winograd_pairs	●	●	●	●				●	需扩展/改写
18 threaded_rows	●	●	●	●		○			需扩展/改写
19 overloaded_operator	●	●	●						需扩展/改写
20 recursive_blocks	●	●	●	●		○			需扩展/改写
21 strassen	●	●	●	●				●	需扩展/改写
22 strassen_winograd	●	●	●	●				●	需扩展/改写
23 csr_dense	●	●	●			○			需扩展/改写
24 csr_csc_merge	●	●	●			○			需扩展/改写
25 coo_scatter	●	●	●	●				●	需扩展/改写

D 数据 · I 索引/控制 · P 任务/结果 · R 显式归约/阶段 · G 图语义 · H 协作 · L 库 · N 数值契约
R 不含普通串行 += ；H 与并行 K 的需求依调度选择。L 标注保留 GPU 库路径时的互操作要求。

功能对照表 3/4：实现 26-32

● 需处理的能力域；○ 常见可选优化。各页区分最低与条件需求；标签不代表当前 GPU 支持。

实现	D	I	P	R	G	H	L	N	状态
26 product_chains					●				链对应版可用
27 dual_cursors			●		●				需扩展/改写
28 row_nodes	●	●	●	●	●	○			需扩展/改写
29 output_cell_tasks	●	●	●		●				需扩展/改写
30 output_tile_tasks	●	●	●		●	○			需扩展/改写
31 bipartite_join	●	●	●		●				需扩展/改写
32 edge_payloads		●	●		●				需扩展/改写

D 数据 · I 索引/控制 · P 任务/结果 · R 显式归约/阶段 · G 图语义 · H 协作 · L 库 · N 数值契约
R 不含普通串行 += ；H 与并行 K 的需求依调度选择。L 标注保留 GPU 库路径时的互操作要求。

功能对照表 4/4：实现 33-40

● 需处理的能力域；○ 常见可选优化。各页区分最低与条件需求；标签不代表当前 GPU 支持。

实现	D	I	P	R	G	H	L	N	状态
33 numpy_operator	●		●				●		GPU 库/扩展
34 numpy_matmul	●		●				●		GPU 库/扩展
35 numpy_dot	●		●				●		GPU 库/扩展
36 numpy_einsum	●		●				●		GPU 库/扩展
37 numpy_tensordot	●		●				●		GPU 库/扩展
38 numpy_broadcast	●	●	●	●			●		GPU 库/扩展
39 scipy_blas	●		●				●		GPU 库/扩展
40 scipy_sparse	●		●	●			●		GPU 库/扩展

D 数据 · I 索引/控制 · P 任务/结果 · R 显式归约/阶段 · G 图语义 · H 协作 · L 库 · N 数值契约
R 不含普通串行 += ；H 与并行 K 的需求依调度选择。L 标注保留 GPU 库路径时的互操作要求。

优先顺序：先打通通用数据与任务，再扩展

第一阶段：最少功能覆盖更多实现

- D：新增固定 dtype 的设备 buffer/view；读写权限、shape/stride 和驻留生命周期。
- I/P：一般循环与下标、int metadata + float 数据、计算型 entry、输出唯一写入、lane 生成坐标。
- 首批验证 flat_buffers / loop_ijk / output_cell_tasks，再到 blocked / register_2x2。
- 保留现有链式路径作正确性对照，分别测构图、打包、上传、kernel 与回传。

后续分支：按产品目标选择

- OSP 表达力：G 的 node handles、第二 receiver、query iterator 与 edge payload，优先 dual_cursors。
- dense 性能：H 的协作 tile / shared memory / barrier / registers；R 支持 split-K 和阶段依赖。
- 快速接成熟算子：L 的 cuBLAS / cuSPARSE 互操作和 resident tensor ABI。
- N：每增加数值优化即补精度/确定性验证，不依赖 CPU 用例自动证明 GPU 正确。

优先级是工程建议；这份报告未实现上述新增 API，也未对 40 种路径做 GPU 性能排名。

如何保留 **OSP** 直觉，同时控制 **CPU** 开销

建议的语义分层

- node：逻辑数据元素或任务坐标；由 view/shape 描述，不要求逐个创建 host 对象。
- edge：关系，可由显式 CSR 或规则 row/column 邻接函数表达；query lowering 隐藏物理表示。
- walker：独立输出/查询的状态与程序；调度策略决定映射到 thread、warp 或 block。

两个具体、可评审的设计方向

- 共享标量 + 双游标：保持 here / rhs 的 graph 语言，但让 handles 与 row/column views 驻留 GPU。
- 共享矩阵 + tile task：任务只携带起点/尺寸，view 提供细粒度索引；块内协作是独立后端契约。
- 先明确同一输出的所有权、布局映射与查询顺序，再讨论自动 reorder；不把大节点当唯一入口。

这些是设计建议；不是已经存在的 Jac tensor/implicit-edge 语法，不能直接复制为可运行程序。

下一轮验证：正确性、开销与访存分别测

正确性清单

- 重用 CPU 19 案例 + 90 参数检查；扩展较大非方阵、空尾块、长 K、稀疏不均匀与抵消数据。
- 比对 CPU/GPU 输出、输入不变、结果索引正确；为混合 index/value 和 bounds 定义失败行为。
- 分别检查逐位稳定要求、容差要求与原子非确定性；Kahan / fsum / Strassen 单独验证。

性能与布局证据

- 分别记录 host 建图/规划、pack、H2D、GPU kernel、D2H；报告 resident 与端到端两种时间。
- 预热 JIT，多次运行；比较同 dtype/shape/layout 的 GPU 库基线，而不是 CPU thread 数。
- 通过 profiler 检查实际 memory transactions、吞吐、shared reuse、占用和分歧；不用示意图替代数据。

现有 validation_results.json 的 elapsed_ms 含准备工作，仅是 CPU 记录，不是 GPU speedup 证据。

资料与源码：报告可回到具体实现

外部资料支持硬件/库/数值语义；逐项缺口来自当前仓库源码检查。所有下划线链接均可点击。

40 项注册表与说明

[jac/examples/matmul/catalog.jac](#)

[jac/examples/matmul/README.md](#)

稠密 / 递归 / 稀疏

[jac/examples/matmul/dense.jac](#)

[jac/examples/matmul/recursive.jac](#)

七种 **OSP** 模型

[jac/examples/matmul/graphs.jac](#)

[jac/examples/matmul/sparse.jac](#)

CPU 库包装与验证证据

[jac/examples/matmul/libraries.jac](#)

[jac/examples/matmul/validation_results.json](#)

GPU walker 选择器 / **CSR**

[jac/jaclang/compiler/backends/native/ptx_walker.jac](#)

[jac/jaclang/compiler/backends/native/ptx_csr.jac](#)

GPU runtime 与链式 **matmul**

[scripts/gpu_walker/runtime.md](#)

[jac/examples/gpu/matmul.jac](#)

外部参考：GPU 原语与算法公式

官方文档 / 原始论文；页面访问日期 2026-10-02。API 可用性仍以目标机器安装版本为准。

CUDA	NVIDIA CUDA Best Practices：合并访存 docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#coalesced-access-to-global-memory
CUTLASS	NVIDIA CUTLASS：分层 GEMM / split-K docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html
CUBLAS	NVIDIA cuBLAS：GEMM 接口 docs.nvidia.com/cuda/cublas/index.html#cublas-t-gemm
CUSPARSE	NVIDIA cuSPARSE：SpMM / SpGEMM docs.nvidia.com/cuda/cusparsedocs/index.html
STRASSEN	Kouya 2014：Strassen / Winograd 公式 arxiv.org/abs/1410.1599
FSUM	Python math.fsum：求和语义 docs.python.org/3/library/math.html#math.fsum
NUMPY	NumPy：matmul / dot / einsum / tensordot numpy.matmulnumpy.dotnumpy.einsumnumpy.tensordot
FLOAT	NVIDIA：浮点舍入、FMA 与结果差异 docs.nvidia.com/cuda/floating-point/index.html

功能解释附录：从术语到代码与后端责任

每项说明：定义、具体例子、编译器/运行时要求、当前支持状态、如何验证；原 40 项说明仍在第 11-50 页。

D1	设备 buffer、dtype 与数组字段	64	G2	query receiver、iterator 与 visit 语义	75
D2	view、shape、stride 与真正的数据转置	65	G3	edge payload、异构节点与 SoA schema	76
D3	设备驻留、分配复用与 workspace 生命周期	66	G4	隐式 node/edge：保留关系，减少实体构图	77
I1	整数寻址、浮点计算与边界掩码	67	H1	thread、warp、block 与协作同步	78
I2	一般循环、局部变量、helper 与语法 lowering	68	H2	global/shared/register/local 与布局选择	79
P1	工作项映射与输出所有权	69	H3	异步复制、流水线与 MMA / Tensor Core	80
P2	计算型 entry、向量输出与 report 的区别	70	L1	GPU 库 FFI、tensor ABI 与 context/stream	81
R1	serial sum、并行归约与原子累加	71	L2	SpMM、SpGEMM 与稀疏 descriptor/workspace	82
R2	多 kernel 阶段与依赖调度	72	N1	精度、FMA 与 reassociation 的契约	83
R3	稀疏结果生成：scan、compaction、分段归约	73	N2	求和顺序、fsum 兼容与确定性	84
G1	node handle、远端属性和第二游标	74			

这些是功能定义和迁移建议。CPU 已有语法 ≠ GPU 已支持；计算示意不代表新增 Jac API。

D1 设备 **buffer**、**dtype** 与数组字段

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- **buffer** 是 GPU 可读写的连续存储，含元素类型与长度；Python list-of-lists 不能仅靠复制对象地址成为设备数组。
- 数组字段应携带设备句柄和元数据，多个 walker 引用同一 A/B；C 的写权限需要明确。
- 这与 fixed obj flattening 不同：后者只把有限个标量成员拆成列，不提供任意长度下标访问。

具体例子（语义示意；不是新 API）

```
A_data: float64[M*K] # shared read-only
B_data: float64[K*N] # shared read-only
C_data: float64[M*N] # owned output
load(A_data, i*K+k); store(C_data, i*N+j, v)
```

用在哪些实现

flat_buffers；六种循环；row_nodes / output_cell_tasks / output_tile_tasks

Jac 要增加什么 / 由哪一层负责

- 语言/类型系统：固定 dtype 的 buffer/view 类型，支持作为 node/walker 字段或 kernel 参数。
- 编译器：设备指针 ABI 与 indexed load/store；运行时：分配、传入句柄、释放及长度验证。
- 最低不要求 GPU 内动态 malloc：C 与临时数组可在 host 预分配，再交给 kernel。

当前 GPU 支持情况

已有 SoA 分配与标量字段；通用设备数组字段和 indexed load/store 尚缺。

怎样检查这项能力

检查多个 walker 共享同一输入缓冲区，而非复制整个矩阵；数组访问结果与 CPU 一致。

D2 view、shape、stride 与真正的数据转置

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- view 描述同一底层 buffer 的一个逻辑窗口：base、offset、shape、stride；不必复制子矩阵。
- 例：2×3 的 A，stride=(3,1)。转置 view 为 3×2、stride=(1,3)，存储次序没有改变。
- 因此 transposed_dot 要获得 BT 行内连续的 k 访问，需要物理转置/打包；只改 view 不会产生该布局收益。

具体例子（语义示意；不是新 API）

```
address(i,j) = base + 8*(offset+i*s0+j*s1)
A:  shape=(2,3), stride=(3,1)
AT: shape=(3,2), stride=(1,3) # same buffer
packed_AT = transpose_copy(A) # reordered buffer
```

用在哪些实现

transposed_dot；blocked / packed_tiles；recursive_blocks；tensordot

Jac 要增加什么 / 由哪一层负责

- 语言：可表达 slice/transpose 的设备 view，明确 alias 与只读/ 可写关系。
- 编译器：把动态 stride/offset 降为整数地址计算，必要时选择连续布局的专门化。
- 运行时或 kernel：需要连续转置结果时执行真实搬运；host 转置也能满足最低正确性要求。

当前 GPU 支持情况

CPU Jac 支持现有列表转置/切片；GPU walker 后端尚无 tensor view。

怎样检查这项能力

验证 transpose view 的坐标映射与底层共享；另验证物理转置后的 stride 与尾块。

D3 设备驻留、分配复用与 workspace 生命周期

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 分配复用是再次使用同一块显存；设备驻留是输入/中间结果保持在 GPU，不反复上传下载。
- 当前 runtime 复用 arena 容量，但每次 run_walkers 仍重新打包/上传，二者不能混为一谈。
- workspace 是 partial、打包 tile 或递归临时矩阵的存储；可以由 host 按最大需要预分配。

具体例子（语义示意；不是新 API）

```
upload(A,B) once
  launch(stage1, A,B,workspace)
  launch(stage2, workspace,C)
download(C) when host needs it
```

用在哪些实现

所有共享矩阵方案；split_k / Strassen；GPU 库路径

Jac 要增加什么 / 由哪一层负责

- 运行时：buffer ownership、跨调用生命周期、输入更新/失效规则与释放时机。
- 阶段结束前不能释放或覆盖仍被 GPU 使用的 workspace；异步执行需完成事件或同步。
- 最低 GPU 运行可每次上传；驻留主要减少端到端开销，属于独立的运行时能力。

当前 GPU 支持情况

已有 arena 容量复用；通用 resident tensor 与输入缓存契约尚缺。

怎样检查这项能力

测试输入更新后结果不会读到旧值；分别测首次上传和多次驻留调用。

I1 整数寻址、浮点计算与边界掩码

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- i/j/k、长度与 offsets 是整数；A/B/C 值及 total 是 float64。同一个 kernel 必须能同时处理两类数据。
- 这不是要求把整数和浮点放进同一数组，而是解除整个 node/walker numeric schema 只能选择一种类型的限制。
- 尾块 mask 在读取/写入前判断边界；不能先越界 load，再把结果乘以 0。

具体例子（语义示意；不是新 API）

```
idx: int64 = i*K+k
value: float64 = A[idx]
valid = (i+1 < M)
a1 = A[(i+1)*K+k] if valid else 0.0
```

用在哪些实现

flat_buffers；坐标/tile 任务；稀疏索引；奇数维 register_2x2

Jac 要增加什么 / 由哪一层负责

- 类型系统/ABI：分别标记 index、value 与 bool；按字段保留各自类型。
- 编译器：整数地址计算与浮点乘加分开，支持动态条件、必要的显式 cast 与有界访问。
- 运行时：验证 shape/stride/容量与索引范围；报错或掩码策略要有明确契约。

当前 GPU 支持情况

已有 bool 条件与 int64/float64 标量；现有 walker 不能混合两种 numeric leaf 类型。

怎样检查这项能力

覆盖 M/N/K 的奇数尺寸和尾块；检查 mask 保护了访问本身，而不仅保护最终 store。

I2 一般循环、局部变量、helper 与语法 lowering

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 当前 GPU traversal 内部会循环，但不等于用户 ability 中可以写任意 for/while；这需要新的 AST lowering。
- 局部 total/k/p/q 要跨迭代保存。range/len/min 可从 buffer 元数据展开，不必在设备调用 Python 对象方法。
- sum/zip/推导式、__matmul__ 或 helper 要静态解析/内联/降为设备操作；host 递归规划不要求 device recursion。

具体例子（语义示意；不是新 API）

```
for k in range(K): total += A[i,k]*B[k,j]
while p < row_end and q < col_end: ...
dot(row,col) -> inline / device function
recursive_plan(...) -> host task list is allowed
```

用在哪些实现

六种循环；csr_csc_merge；comprehension；
overloaded_operator；递归算法

Jac 要增加什么 / 由哪一层负责

- 编译器：循环控制流、跨迭代 SSA 值、局部赋值、动态比较与 return 的类型化处理。
- 静态 helper 可内联或编译成 device function；动态 Python callable / generator 不能直接送入 PTX。
- 只需支持计算主体用到的语法；输入检查、目录注册和 CPU wrapper 可以继续留在 host 执行。

当前 GPU 支持情况

当前支持私有 self 字段赋值与 if；一般用户循环、局部变量和函数调用均未完整支持。

怎样检查这项能力

检查循环步数/停止条件、零次循环、双指针推进和 helper 的参数类型。

P1 工作项映射与输出所有权

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- GPU 工作项回答“哪个线程算哪个结果”。最小方案是每个线程独占一个 $C[i,j]$ ，串行遍历 K 。
- 输出唯一所有权可免原子；若把不同 k 分给多个工作项，它们不能无规则地同时修改同一 C 。
- 任务坐标可由线程编号生成，无需 CPU 逐个构建 MN 个 `CellTask`；逻辑任务与实体对象可以分离。

具体例子（语义示意；不是新 API）

```
t = global_thread_id
i = t // N; j = t % N
if t < M*N: C[i,j] = serial_dot(A,B,i,j)
# each C[i,j] has exactly one writer
```

用在哪些实现

所有 dense 方案；`output_cell_tasks` / `output_tile_tasks`；`threaded_rows`

Jac 要增加什么 / 由哪一层负责

- 编译器/调度：指定 grid、工作项到 i/j 的映射，以及任务边界保护。
- 语言/运行时：定义私有 state、共享只读输入、独占结果 view；串行代码不会自动安全并行。
- 若保留真实 walker 对象，host 仍可 batch 提交；隐式任务空间是减少 host 对象的额外能力。

当前 GPU 支持情况

已有一线程一独立 walker 的 batch；没有通用坐标工作项或矩阵输出 ABI。

怎样检查这项能力

确认每个输出恰好一次被写入；测试小 N 、非整 block 尾部及不同任务排序。

P2 计算型 entry、向量输出与 report 的区别

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 这些 entry 到达任务节点后立即计算，无需继续走图；当前选择器要求能力末尾有一个 tail visit。
- 单点任务可通过补一个空后继 typed visit 适配现有控制形式；这仍不能绕过数组/查询等其他缺口。
- N 个输出可写入预分配的私有向量/view。标量 report 流有顺序与容量语义，不自动等价于 tensor 输出。

具体例子（语义示意；不是新 API）

```
entry CellTask:
    total = dot(A[row,:], B[:,col])
    finish_without_visit
entry TileTask: store_owned_tile(C, output)
```

用在哪些实现

row_nodes ; output_cell_tasks / output_tile_tasks ; edge_payloads

Jac 要增加什么 / 由哪一层负责

- 编译器：允许有限的 compute entry 正常结束，或明确采用空 tail-visit 的适配方式。
- ABI/运行时：标量、向量/tile 输出的长度、所有权和回写方式；避免每线程构造 Python list。
- 如用 report 表达结果，须定义 slot 到坐标的映射、类型与容量，不能仅凭现有 report buffer 推断数组支持。

当前 GPU 支持情况

已有标量 state/report；无 visit entry 与一般向量/矩阵结果还未完整实现。

怎样检查这项能力

验证只执行一次 entry、结果尺寸正确；区分更新 walker 私有字段与共享矩阵 store。

R1 serial sum、并行归约与原子累加

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 归约把多个贡献合成一个输出。串行 `total +=` 不需要线程通信；并行 `K` 或 `scatter` 才产生跨工作项合并问题。
- 树形归约可以用分组部分和，再合并；原子累加则把多个并行更新串行化到同一目标位置。
- 原子保证更新不会丢失，但浮点执行顺序可能不同；固定树也必须明确分组，才能讨论确定性。

具体例子（语义示意；不是新 API）

```
serial: C = (((p0+p1)+p2)+p3)
tree:   C = (p0+p1) + (p2+p3)
split:  C = reduce(partial[chunk,:])
scatter: atomic_add(C[i,j], contribution)
```

用在哪些实现

`pairwise` ; `split_k` ; `comprehension` ; `coo_scatter` ; 稀疏行内协作

Jac 要增加什么 / 由哪一层负责

- 编译器：显式 `reduce` 的运算符、单位元、轴/分段和求和顺序；可选 `shuffle/shared` 降低。
- 运行时：跨 kernel partial buffer 和归约阶段；原子路径需支持目标 `dtype` 的 `atomic` 操作。
- 最小 `matmul` 保持每输出一个所有者、串行 `K`，就可以不实现跨线程归约。

当前 GPU 支持情况

已有串行 `walker` 私有累加；通用 `reduce`、`shuffle` 与共享输出 `atomics` 尚缺。

怎样检查这项能力

测试并行贡献没有丢失；再分别检查数学容差、固定顺序及逐位确定性。

R2 多 kernel 阶段与依赖调度

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 阶段依赖表示必须先产生中间结果，再由后续 kernel 消费；例如 partial→reduce，行/列因子→主公式。
- Strassen 的递归可以由 CPU 规划成子任务图，子矩阵和中间结果保持在 GPU。
- barrier 只解决协作范围内的同步；不能把一个 block 的 barrier 当作整个 grid 的阶段边界。

具体例子（语义示意；不是新 API）

```
plan on host
  launch(make_partials, workspace)
  after completion: launch(reduce_partials, workspace, C)
  reuse workspace only after all consumers finish
```

用在哪些实现

split_k；winograd_pairs；Strassen / strassen_winograd；GPU 转置

Jac 要增加什么 / 由哪一层负责

- 运行时：有序 kernel 提交/完成语义，workspace 分配、引用生命周期、错误传播。
- 跨 stream 执行要有事件依赖；同步提交或同一有序 stream 可先满足最低要求。
- 不必先支持 GPU 发起 kernel launch，也不必把整个 Python/Jac 递归调用栈搬到设备。

当前 GPU 支持情况

现有 runtime 是单个 walker batch 路径；没有通用矩阵算子 DAG/多阶段调度接口。

怎样检查这项能力

验证消费阶段不会读未完成的中间值；分别统计规划、复制、启动和kernel 时间。

R3 稀疏结果生成：scan、compaction、分段归约

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 稀疏×稀疏会产生未知数量的位置；必须合并同一 (i,j) 的贡献，并确定最终 nnz 与 offsets。
- scan 是前缀和，用数量/保留标记计算写入位置；compaction 按这些位置保留有效项。
- 排序/哈希 join、分段归约与容量规划是一组结构算子；不是把图 CSR 上传后即可得到的能力。

具体例子（语义示意；不是新 API）

```
contributions -> group by (i,j) -> sum duplicates
counts_per_row -> prefix_sum -> output_offsets
allocate capacity -> compact values/indices
optionally sparse_to_dense(C)
```

用在哪些实现

scipy_sparse 的 native 替代路径；并行 coo_scatter 的有序替代

Jac 要增加什么 / 由哪一层负责

- native 路线：D/I/R 的结构生成、去重数值归约、边界/容量策略与多阶段依赖。
- 库路线：由 cuSPARSE 完成相应阶段，Jac 负责 descriptor、workspace、结果分配与调用契约。
- 如果 CPU 预处理已提供固定结构，最低 GPU kernel 可只做数值阶段；须明确哪部分仍在 host。

当前 GPU 支持情况

当前图 traversal 没有稀疏矩阵结果结构生成；GPU 库路线也尚未接入。

怎样检查这项能力

检验重复贡献合并、空行、输出 nnz/offsets 和最终 dense 结果，不能只检查总和。

G1 node handle、远端属性和第二游标

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- node handle 是设备可解析的节点身份，如“节点类型 + 索引”；它不是 host 对象地址，也不是复制一份节点值。
- here 已由当前遍历位置定位；self.rhs 需要独立定位另一节点，再从对应属性列读取 value。
- 图重排时必须同时重映射 here/heads、rhs 等所有句柄；packer 也要收集第二游标可达的图。

具体例子（语义示意；不是新 API）

```
self.total += here.value * self.rhs.value;
following = [self.rhs->:ColumnNext:->];
self.rhs = following[0];
visit [->:RowNext:->];
```

用在哪些实现

dual_cursors；bipartite_join 的远端节点访问

Jac 要增加什么 / 由哪一层负责

- 类型/编译器：节点引用状态、类型化 handle、远端字段 load 和 handle 赋值。
- 运行时：多根打包、引用闭包、索引 remap、有效性/类型验证；共享节点保留同一身份。
- 固定 plain obj 的 flattening 是值字段展开，不能直接代替 node reference 与 alias 语义。

当前 GPU 支持情况

here 标量列读取已有；self.rhs 节点状态与远端解引用在现有 GPU 子集中不支持。

怎样检查这项能力

重排后双游标仍匹配同一 k；两个 walker 引用相同输入节点时不产生副本或身份错误。

G2 query receiver、iterator 与 visit 语义

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- visit 负责安排下一次 entry；普通 query 负责返回/枚举关系结果，可以在当前 entry 中立即读取和累加。
- 第二 receiver query 从 rhs 出发；neighbor iterator 从 here 出发；edge query 枚举边实例而不是目标节点。
- CSR 是存储格式：offsets 给区间，targets 给下一跳。它不自动提供过滤、多个边类型或 Jac 查询结果的完整语义。

具体例子（语义示意；不是新 API）

```
query(rhs, ColumnNext) -> handle/result
  for other in query(here, SameK): read(other.value)
  for edge in edge_query(here, Factor): read(edge.left)
  visit(...) -> schedule later entry
```

用在哪些实现

dual_cursors；bipartite_join；edge_payloads

Jac 要增加什么 / 由哪一层负责

- 编译器：区分 query 与 visit，降低 receiver、iterator、索引/空结果和属性访问。
- 数据/运行时：每种关系的 CSR/视图描述、查询顺序；节点结果与多边实例的去重规则分别定义。
- 最低可先支持代码中用到的单跳无过滤 query；任意过滤和嵌套查询不是首次迁移的前提。

当前 GPU 支持情况

当前支持单一 typed tail visit；CSR FIFO 可分支，但一般 query/edge iterator 未支持。

怎样检查这项能力

覆盖空邻居、多邻居、第二 receiver 和重复到达；核对顺序与 node/edge 结果类型。

G3 edge payload、异构节点与 SoA schema

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- edge payload 是边自己的属性：Factor.left/right 要按边索引读，不能从 targets 的目标节点列推断。
- bipartite_join 有 LeftValue 与 RightValue 两类节点；RightValue 同时保存 float value 和 int column。
- SoA 可以按节点/边类型建立独立字段列；逻辑 schema、物理列与 handle 类型必须一致。

具体例子（语义示意；不是新 API）

```
edge arena: left[E] | right[E] | targets[E]
right-node arena: value[NB] | column[NB]
edge_dot: sum(edge.left * edge.right)
join: output[other.column] += here.value*other.value
```

用在哪些实现

edge_payloads；bipartite_join；带数值权重的图算法

Jac 要增加什么 / 由哪一层负责

- 编译器：类型化 edge handle/field load、异构 node 属性访问、按字段 dtype 选择指令。
- packer/ABI：边索引重排同步搬运所有属性；多个 node schema 的身份和列布局。
- 这些例子只读图属性，无需先实现共享图修改或并发增删边。

当前 GPU 支持情况

当前 tail-visit edge 必须为空属性；触发/属性访问与 numeric schema 也受到单一类型限制。

怎样检查这项能力

检查 edge 属性跟随正确边，而不是随目标节点错位；验证 column 索引与 float value 独立。

G4 隐式 node/edge：保留关系，减少实体构图

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 规则矩阵可以用 shape 与坐标定义逻辑节点；row-next/column-next 可由坐标函数算出，不必保存所有实体边。
- 任务节点也可只是 (i,j) 或 tile 起点；CPU 不必逐个创建对象。与一般不规则 CSR 图可以共存。
- 这是建议的能力：需要定义节点身份、关系查询、终止与物理布局，不能只改存储后假设现有 query 自动适用。

具体例子（语义示意；不是新 API）

```
A-node identity = (matrix_id, i, k)
RowNext(i,k) = (i,k+1) if k+1<K else none
B-node identity = (matrix_id, k, j)
CellTask(t) = (t//N, t%N)
```

用在哪些实现

dual_cursors ; output_cell_tasks / output_tile_tasks ; bipartite_join

Jac 要增加什么 / 由哪一层负责

- 语言/IR：逻辑 node/edge view 或任务空间；区分身份与对象实体，不承诺已有新语法。
- 编译器：关系查询降为索引计算，保证矩阵 view 的读写与终止语义。
- 运行时：shape/stride 参数与输入 buffer；适用规则关系，不强求所有图都隐式化。

当前 GPU 支持情况

当前 packer 处理真实 Jac 对象和显式拓扑；尚无规则图 view 的 GPU lowering。

怎样检查这项能力

对照实体图检查可达节点/顺序/输出；再测 CPU 准备成本是否从 MNK 构图降下去。

H1 thread、warp、block 与协作同步

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 当前一个 walker 由一个 GPU thread 执行；把逻辑 walker 改成 warp/block 任务，需要新的参与与分工契约。
- 各线程可以协作加载一个 tile，再等待数据就绪，分别持有自己负责的结果片段。
- 参与 barrier 的线程必须按规定到达；边界线程可以掩码数据，但不能任意退出后让其他线程等待。

具体例子（语义示意；不是新 API）

```
cooperative_load(A_tile,B_tile)
  barrier_for_participants()
  each_thread_accumulates_its_output_fragment()
  barrier_before_reusing_tile_storage()
```

用在哪些实现

blocked / packed_tiles ; output_tile_tasks ; 行/点积的协作版本

Jac 要增加什么 / 由哪一层负责

- 调度/语言：协作范围、参与线程、共享 scratch 与输出片段所有权。
- 编译器：正确的同步 lowering，检查分支/循环与参与范围；运行时校验 launch 资源。
- 这是高性能扩展：一线程独占输出、串行 K 的最低正确版本不要求barrier。

当前 GPU 支持情况

现有 walker 状态完全私有，未提供 block/warp 协作和 barrier 的用户语义。

怎样检查这项能力

验证部分尾块、参与掩码与 tile 缓冲复用；不能只比较正常整块输出。

H2 global/shared/register/local 与布局选择

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- global 存整个矩阵；shared 是协作范围内的 tile 缓存；寄存器保存每线程的标量/累加器。
- private/local 不保证数据就在寄存器：动态数组或寄存器压力可能导致内存访问。
- global coalescing 关注相邻 lane 的地址；shared bank conflict 是另一层问题，可用 padding/swizzle 调整。

具体例子（语义示意；不是新 API）

```
global A/B -> shared tile -> per-thread accumulators
lane-j load: B[k,j+lane] # neighboring addresses
output: store fragments to owned C positions
layout may change between memory spaces
```

用在哪些实现

packed_tiles；register_2x2；GPU 分块；coalescing 优化

Jac 要增加什么 / 由哪一层负责

- 语言/IR：地址空间、共享 scratch 与固定/有界寄存器片段；明确谁能访问哪段存储。
- 编译器：layout/access-map 选择、协作 pack、寄存器分配；不能承诺普通 list 自动变 shared。
- 性能调优：同时评估访问连续性、复用、占用和 spill；SoA 本身不能保证任意下一跳 coalescing。

当前 GPU 支持情况

已有 global SoA 与线程私有标量；显式 shared memory 与协作布局尚缺。

怎样检查这项能力

用 profiler 检查实际 transactions、bank conflicts 与 spill，不能用变量名推断内存位置。

H3 异步复制、流水线与 MMA / Tensor Core

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- 流水线让当前 tile 计算与下一 tile 的加载重叠；必须跟踪哪些缓冲可读、何时可复用。
- MMA 把矩阵片段乘加映射到硬件指令；需要目标支持的 dtype、形状、布局和协作规则。
- 这些均是可选性能能力；不能为获得 Tensor Core 而未经声明把 float64 样例改成低精度。

具体例子（语义示意；不是新 API）

```
load(next_stage) while compute(current_stage)
wait_until_stage_ready(); consume(stage)
optional: accumulator = mma(fragment_A, fragment_B)
fallback: scalar/SIMT multiply-add
```

用在哪些实现

高性能 blocked / packed_tiles / output_tile_tasks；可选 GEMM 后端

Jac 要增加什么 / 由哪一层负责

- 编译器：流水线依赖、等待、缓冲轮换，或专门的 GEMM/MMA intrinsic lowering。
- 后端：按目标架构/数据类型选择可用指令并提供 fallback；库路线可让库内部完成优化。
- 运行时/数值契约：检查资源、能力与精度模式；性能目标不能改变结果契约而不告知。

当前 GPU 支持情况

现有 scalar walker 未实现异步 tile 复制、MMA 片段或这类协作流水线。

怎样检查这项能力

先验证 dtype/形状和结果误差，再测重叠是否减少耗时；不支持的硬件应有明确行为。

L1 GPU 库 FFI、tensor ABI 与 context/stream

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- Jac 在 host 调用 GPU 库，库在 GPU 发 kernel；这种互操作与把 Jac 计算主体编译成 PTX 是两条路线。
- GPU tensor ABI 需提供设备指针、dtype、shape/stride；库 handle 与 Jac 的 CUDA context/stream 要协调。
- 当前 NumPy/SciPy 包装调用的是 CPU 库。普通 import 或 CPU BLAS 调用不能直接放入 walker 设备函数。

具体例子（语义示意；不是新 API）

```
host Jac: obtain device A/B/C pointers
host FFI: call GPU GEMM(handle, dims, pointers)
GPU library: execute kernel
host Jac: synchronize/check status before reading C
```

用在哪些实现

numpy_operator / matmul / dot / einsum / tensordot；scipy_blas

Jac 要增加什么 / 由哪一层负责

- 运行时：GPU 库 wrapper/FFI、可用性检查、handle/context/stream 生命周期及错误管理。
- 数据适配：row/column-major、leading dimension 与 transpose；不兼容布局需 view 调整或 pack。
- 如走 compiler intrinsic 路线，可把已识别的算子直接 lower；并不要求完整复制 NumPy API。

当前 GPU 支持情况

现有 host ctypes CUDA Driver API 是基础；没有 cuBLAS tensor/handle 的整合接口。

怎样检查这项能力

检验库实际使用 device pointers、无意外 CPU fallback，且输出与 float64 CPU 参考一致。

L2 SpMM、SpGEMM 与稀疏 descriptor/workspace

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- SpMM 是稀疏×稠密，C 形状已知；SpGEMM 是稀疏×稀疏，结果结构和 nnz 也需要产生。
- descriptor 说明索引/值缓冲、shape、index 类型与格式；workspace 为库算法的临时存储。
- Jac 图 CSR、数值矩阵 CSR 和库 descriptor 可以共享部分表示思路，但查询/数值/结构生成语义不同。

具体例子（语义示意；不是新 API）

```
SpMM: CSR(A) + dense(B) -> dense(C)
SpGEMM: CSR(A) + CSR(B) -> CSR(C)
workflow: describe -> workspace/structure -> compute
scipy_sparse sample then converts C to dense
```

用在哪些实现

csr_dense 的库替代；scipy_sparse；稀疏 GPU 后端

Jac 要增加什么 / 由哪一层负责

- 库适配/运行时：descriptor 与缓冲区绑定、调用所需阶段、容量/workspace 与释放时机。
- 形状/类型：正确 index base、索引宽度和 float64；最终 dense 转换也要定义在哪执行。
- 不必自己实现库内部 scan/排序/归约；但不能忽略结果结构分配和阶段完成规则。

当前 GPU 支持情况

当前未接 GPU 稀疏库；既有 CSR traversal 不等价于库 SpMM/SpGEMM。

怎样检查这项能力

分别测试空行/不同 nnz/结果结构与 dense 输出；兼容性以目标机器的库版本为准。

N1 精度、FMA 与 reassociation 的契约

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- float64 表示输入和累加的精度；若改用更低精度计算，必须作为另一种数值模式验证。
- FMA 对乘加一起舍入，与先乘再加的舍入路径不同；重排求和也会改变结果。
- Kahan 的 correction 依赖表达式的具体括号；激进的代数重组可能把补偿效果消除。

具体例子（语义示意；不是新 API）

```
separate: p = round(a*b); s = round(s+p)
fused:    s = round(a*b+s)
Kahan: correction = (updated-total)-adjusted
preserve declared dtype and numerical mode
```

用在哪些实现

所有 float64 路径；compensated；Winograd / Strassen；MMA 后端

Jac 要增加什么 / 由哪一层负责

- 语言/编译器：明确 strict / contracted / relaxed 等模式的语义，再决定是否融合或重组。
- 后端/库：选择匹配 dtype 与模式的计算路径；不把更快但不同精度称为相同验证。
- 运行/验证：记录所用精度、编译选项与参考容差；逐位一致要求需单独说明。

当前 GPU 支持情况

当前样例与状态使用 float64；没有通用 tensor/MMA 数值模式接口。

怎样检查这项能力

加入抵消、量级差和非方阵数据；同时验证模式选择和实际 kernel/库的计算类型。

N2 求和顺序、fsum 兼容与确定性

术语展开与设计解释；以下计算示意不构成已实现的新 Jac GPU API。

是什么 / 为什么需要

- pairwise、Kahan、fsum 是不同的求和策略；不能把任何普通 GPU reduce 当作 math.fsum 的兼容实现。
- fsum 跟踪多个部分和；若需要保留该路径，必须提供设备实现/内建并定义兼容精度范围。
- 确定性指多次运行的结果是否稳定；容差一致指接近参考。两者和逐位等于 CPU 是不同要求。

具体例子（语义示意；不是新 API）

```
p = [1e16, 1.0, -1e16]
sequential left fold: 0.0
math.fsum(p): 1.0
fixed grouping != automatically CPU-bitwise equal
```

用在哪些实现

```
pairwise / split_k / compensated / accurate_sum ; coo_scatter
```

Jac 要增加什么 / 由哪一层负责

- 编译器/算法：保留指定分组或补偿算法；并行版本需明确局部计算与最终合并。
- 运行时：固定任务/归约顺序可支持确定性；原子路径不能仅靠 atomic 防丢失来保证舍入顺序。
- 现有 CPU 记录只覆盖列出的 19 案例与容差；新增 GPU 路径必须重新做差分。

当前 GPU 支持情况

scalar fold 已有；tree/Kahan 可有改写路线，fsum helper 和通用并行归约仍缺。

怎样检查这项能力

分别声明数学误差、重复运行稳定性和 CPU 逐位比较目标，不用一个 PASS 混合三种含义。